

CHAPTER 11

Binary Trees and B-Trees

IN THIS CHAPTER, YOU WILL:

- Learn about binary trees
- Explore various binary tree traversal algorithms
- Learn how to organize data in a binary search tree
- Discover how to insert and delete items in a binary search tree
- Explore nonrecursive binary tree traversal algorithms
- Learn about AVL (height-balanced) trees
- Learn about B-trees

When data is being organized, a programmer's highest priority is to organize it in such a way that item insertion, deletion, and lookups (searches) are fast. You have already seen how to store and process data in an array. Because an array is a random access data structure, if the data is properly organized (for example, sorted), we can use a search algorithm, such as a binary search, to effectively find and retrieve an item from the list. However, we know that storing data in an array has its limitations. For example, item insertion (especially if the array is sorted) and item deletion can be very time consuming, especially if the list size is very large, because each of these operations requires data movement. To speed up item insertion and deletion, we can use linked lists. Item insertion and deletion in a linked list do not require any data movement; we simply adjust some of the pointers in the list. However, one of the drawbacks of linked lists is that they must be processed sequentially. That is, to insert or delete an item, or simply search the list for a particular item, we must begin our search at the first node in the list. As you know, a sequential search is good only for very small lists because the average search length of a sequential search is half the size of the list.

Binary Trees

This chapter discusses how to dynamically organize data so that item insertion, deletion, and lookups are more efficient.

We first introduce some definitions to facilitate our discussion.

Definition: A **binary tree**, T, is either empty or such that

i. T has a special node called the **root** node.
ii. T has two sets of nodes, L_T and R_T, called the **left subtree** and **right subtree** of T, respectively.
iii. L_T and R_T are binary trees.

A binary tree can be shown pictorially. Suppose that T is a binary tree with a root node A. Let L_A denote the left subtree of A and R_A denote the right subtree of A. Now L_A and R_A are binary trees. Suppose that B is the root node of L_A and C is the root node of R_A. B is called the **left child** of A; C is called the **right child** of A. Furthermore, A is called the **parent** of B and C.

In the diagram of a binary tree, each node of the binary tree is represented as a circle and the circle is labeled by the node. The root node of the binary tree is drawn at the top. The left child of the root node (if any) is drawn below and to the left of the root node. Similarly, the right child of the root node (if any) is drawn below and to the right of the root node. Children are connected to the parent by an *arrow* from the parent to the child. An arrow is usually called a **directed edge** or a **directed branch** (or simply a **branch**). Because the root node, B, of L_A is already drawn, we apply the same procedure to draw the remaining parts of L_A. R_A is drawn similarly. If a node has no left child, for example, when we draw an arrow from the node to the left child, we end the arrow with three lines. That is, three lines at the end of an arrow indicate that the subtree is empty.

The diagram in Figure 11-1 is an example of a binary tree. The root node of this binary tree is A. The left subtree of the root node, which we denoted by L_A, is the set $L_A = \{B, D, E, G\}$ and the right subtree of the root node, which we denote by R_A, is the set $R_A = \{C, F, H\}$. The root node of the left subtree of A—that is, the root node of L_A—is node B. The root node of R_A is C, and so on. Clearly, L_A and R_A are binary trees. Because three lines at the end of an arrow mean that the subtree is empty, it follows that the left subtree of D is empty.

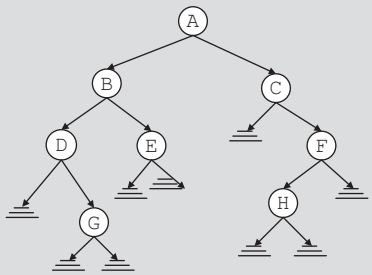


FIGURE 11-1 Binary tree

In Figure 11-1, the left child of A is B and the right child of A is C. Similarly, for node F, the left child is H and node F has no right child.

Example 11-1 shows nonempty binary trees.

EXAMPLE 11-1

Figure 11-2 shows binary trees with one, two, or three nodes.

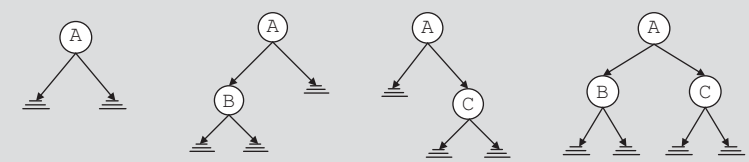


FIGURE 11-2 Binary tree with one, two, or three nodes

In the binary tree of Figure 11-2(a), the root node is A, L_A = empty, and R_A = empty.

In the binary tree of Figure 11-2(b), the root node is A, $L_A = \{B\}$, and R_A = empty. The root node of $L_A = B$, L_B = empty, and R_B = empty.

In the binary tree of Figure 11-2(c), the root node is A, L_A = empty, $R_A = \{C\}$. The root node of $R_A = C$, L_C = empty, and R_C = empty.

In the binary tree of Figure 11-2(d), the root node is A, $L_A = \{B\}$, $R_A = \{C\}$. The root node of $L_A = B$, L_B = empty, R_B = empty. The root node of $R_A = C$, L_C = empty, R_C = empty.

EXAMPLE 11-2

This example shows other cases of nonempty binary trees with three nodes. See Figure 11-3.

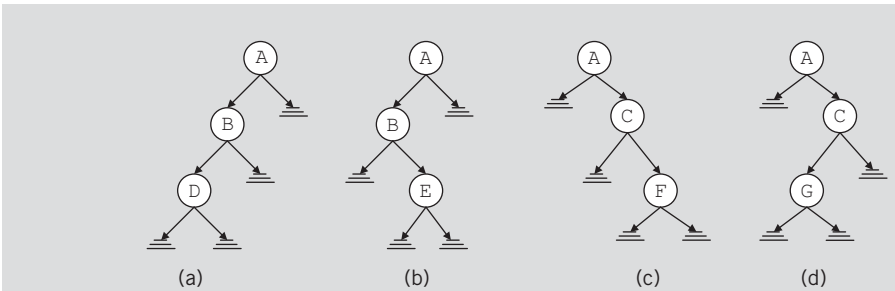


FIGURE 11-3 Various binary trees with three nodes

As you can see from the preceding examples, every node in a binary tree has at most two children. Thus, every node, other than storing its own information, must keep track of its left subtree and right subtree. This implies that every node has two pointers, `llink` and `rlink`. The pointer `llink` points to the root node of the left subtree; the pointer `rlink` points to the root node of the right subtree.

The following `struct` defines the node of a binary tree:

```
template <class elemType>
struct binaryTreeNode
{
    elemType info;
    binaryTreeNode<elemType> *llink;
    binaryTreeNode<elemType> *rlink;
};
```

From the definition of the node, it is clear that for each node,

- The data is stored in `info`.
- A pointer to the left child is stored in `llink`.
- A pointer to the right child is stored in `rlink`.

Furthermore, a pointer to the root node of the binary tree is stored outside the binary tree in a pointer variable, usually called the **root**, of type `binaryTreeNode`. Thus, in general, a binary tree looks like the diagram in Figure 11-4.

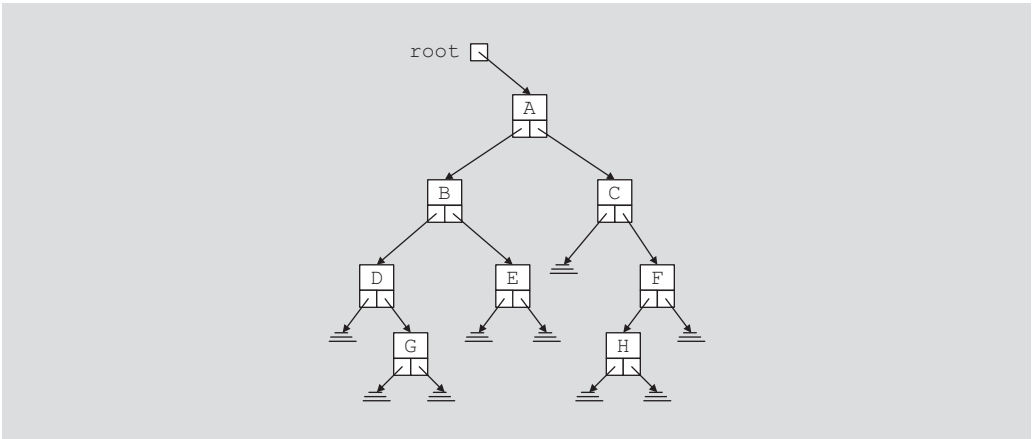


FIGURE 11-4 Binary tree

For simplicity, we will continue to draw binary trees as before. That is, we use circles to represent nodes, and left and right arrows to represent links. As before, three lines at the end of an arrow mean that the subtree is empty.

Before we leave this section, let us define a few more terms.

A node in the binary tree is called a **leaf** if it has no left and right children. Let U and V be two nodes in the binary tree T. U is called a **parent** of V if there is a branch from U to V. A **path** from a node X to a node Y in the binary tree is a sequence of nodes $X_0, X_1, \ldots, X_n$ such that

i. $X = X_0$, $X_n = Y$
ii. X_{i-1} is the parent of X_i for all $i = 1, 2, \ldots, n$. That is, there is a branch from X_0 to X_1, X_1 to $X_2, \ldots, X_{i-1}$ to $X_i, \ldots, X_{n-1}$ to X_n.

Because the branches go only from a parent to its children, from the previous discussion it is clear that in a binary tree, there is a unique path from the root to every node in the binary tree.

Definition: The **level of a node** in a binary tree is the number of branches on the path from the root to the node.

Clearly, the level of the root node of a binary tree is 0, and the level of the children of the root node is 1.

Definition: The **height of a binary tree** is the number of nodes on the longest path from the root to a leaf.

Suppose that a pointer `p` to the root node of a binary tree is given. We next describe the C++ function height to find the height of the binary tree. The pointer to the root node is passed as a parameter to the function height.

If the binary tree is empty, the height is 0. Suppose that the binary tree is nonempty. To find the height of the binary tree, we first find the height of the left subtree and the height of the right subtree. We then take the maximum of these two heights and add 1 to find the height of the binary tree. To find the height of the left (right) subtree, we apply the same procedure because the left (right) subtree is a binary tree. Therefore, the general algorithm to find the height of a binary tree is as follows. Suppose `height(p)` denotes the height of the binary tree with root `p`.

```
if (p is NULL)
    height(p) =  0
else
    height(p) = 1 + max(height(p->llink), height(p->rlink))
```

Clearly, this is a recursive algorithm. The following function implements this algorithm:

```
template <class elemType>
int height(binaryTreeNode<elemType> *p) const
{
    if (p == NULL)
        return 0;
    else
        return 1 + max(height(p->llink), height(p->rlink));
}
```

The definition of the function height uses the function `max` to determine the larger of two integers. The function `max` can be easily implemented.

Similarly, we can implement algorithms to find the number of nodes and number of leaves in a binary tree.

Copy Tree

One useful operation on binary trees is to make an identical copy of a binary tree. A binary tree is a dynamic data structure; that is, memory for its nodes is allocated and deallocated during program execution. Therefore, if we use just the value of the pointer of the root node to make a copy of a binary tree, we get a shallow copy of the data. To make an identical copy of a binary tree, we need to create as many nodes as there are in the binary tree to be copied. Moreover, in the copied tree, these nodes must appear in the same order as they are in the original binary tree.

Given a pointer to the root node of a binary tree, we next describe the function `copyTree`, which makes a copy of a given binary tree. This function is also useful in implementing the copy constructor and overloading the assignment operator, as described later in this chapter (see the section, "Implementing Binary Trees").

```
template <class elemType>
void copyTree(binaryTreeNode<elemType>* &copiedTreeRoot,
              binaryTreeNode<elemType>* otherTreeRoot)
{
    if (otherTreeRoot == NULL)
        copiedTreeRoot = NULL;
```

```
    else
    {
        copiedTreeRoot = new binaryTreeNode<elemType>;
        copiedTreeRoot->info = otherTreeRoot->info;
        copyTree(copiedTreeRoot->llink, otherTreeRoot->llink);
        copyTree(copiedTreeRoot->rlink, otherTreeRoot->rlink);
    }
}//end copyTree
```

Binary Tree Traversal

The item insertion, deletion, and lookup operations require that the binary tree be traversed. Thus, the most common operation performed on a binary tree is to traverse the binary tree, or visit each node of the binary tree. As you can see from the diagram of a binary tree, the traversal must start at the root node because there is a pointer to the root node. For each node, we have two choices:

- Visit the node first.
- Visit the subtrees first.

These choices lead to three different traversals of a binary tree—Inorder, preorder, and postorder.

Inorder Traversal

In an inorder traversal, the binary tree is traversed as follows:

1. Traverse the left subtree.
2. Visit the node.
3. Traverse the right subtree.

Preorder Traversal

In a preorder traversal, the binary tree is traversed as follows:

1. Visit the node.
2. Traverse the left subtree.
3. Traverse the right subtree.

Postorder Traversal

In a postorder traversal, the binary tree is traversed as follows:

1. Traverse the left subtree.
2. Traverse the right subtree.
3. Visit the node.

Clearly, each of these traversal algorithms is recursive.

The listing of the nodes produced by the inorder traversal of a binary tree is called the **inorder sequence**. The listing of the nodes produced by the preorder traversal of a binary tree is called the **preorder sequence**. The listing of the nodes produced by the postorder traversal of a binary tree is called the **postorder sequence**.

Before giving the C++ code for each of these traversals, let us illustrate the inorder traversal of the binary tree in Figure 11-5. For simplicity, we assume that visiting a node means to output the data stored in the node. The section, "Binary Tree Traversal and Functions as Parameters," located later in this chapter, explains how to modify the binary tree traversal algorithms so that by using a function, the user can specify the action to be performed on a node when the node is visited.

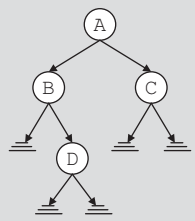


FIGURE 11-5 Binary tree for an inorder traversal

A pointer to the binary tree in Figure 11-5 is stored in the pointer variable `root` (which points to the node with info *A*). Therefore, we start the traversal at *A*.

1. Traverse the left subtree of *A*; that is, traverse $L_A = \{B, D\}$.
2. Visit *A*.
3. Traverse the right subtree of *A*; that is, traverse $R_A = \{C\}$.

Now, we cannot do Step 2 until we have finished Step 1.

1. Traverse the left subtree of *A*; that is, traverse $L_A = \{B, D\}$. Now L_A is a binary tree with the root node *B*. Because L_A is a binary tree, we apply the inorder traversal criteria to L_A.
    1.1. Traverse the left subtree of *B*; that is, traverse $L_B =$ empty.
    1.2. Visit *B*.
    1.3. Traverse the right subtree of *B*; that is, traverse $R_B = \{D\}$.

As before, first we complete Step 1.1 before going to Step 1.2.

    1.1. Because the left subtree of *B* is empty, there is nothing to traverse. Step 1.1 is completed, so we proceed to Step 1.2.
    1.2. Visit *B*. That is, output *B* on an output device. Clearly, the first node printed is *B*. This completes Step 1.2, so we proceed to Step 1.3.

1.3. Traverse the right subtree of B; that is, traverse $R_B = \{D\}$. Now R_B is a binary tree with the root node D. Because R_B is a binary tree, we apply the inorder traversal criteria to R_B.

1.3.1. Traverse the left subtree of D; that is, traverse $L_D =$ empty.

1.3.2. Visit D.

1.3.3. Traverse the right subtree of D; that is, traverse $R_D =$ empty.

1.3.1. Because the left subtree of D is empty, there is nothing to traverse. Step 1.3.1 is completed, so we proceed to Step 1.3.2.

1.3.2. Visit D. That is, output D on an output device. This completes Step 1.3.2, so we proceed to Step 1.3.3.

1.3.3. Because the right subtree of D is empty, there is nothing to traverse. Step 1.3.3 is completed.

This completes Step 1.3. Because Steps 1.1, 1.2, and 1.3 are completed, Step 1 is completed, and so we go to Step 2.

2. Visit A. That is, output A on an output device. This completes Step 2, so we proceed to Step 3.
3. Traverse the right subtree of A; that is, traverse $R_A = \{C\}$. Now R_A is a binary tree with the root node C. Because R_A is a binary tree, we apply the inorder traversal criteria to R_A.

3.1. Traverse the left subtree of C; that is, traverse $L_C =$ empty.

3.2. Visit C.

3.3. Traverse the right subtree of C; that is, traverse $R_C =$ empty.

3.1. Because the left subtree of C is empty, there is nothing to traverse. Step 3.1 is completed.

3.2. Visit C. That is, output C on an output device. This completes Step 3.2, so we proceed to Step 3.3.

3.3. Because the right subtree of C is empty, there is nothing to traverse. Step 3.3 is completed.

This completes Step 3, which in turn completes the traversal of the binary tree.

Clearly, the inorder traversal of the previous binary tree outputs the nodes in the following order:

Inorder sequence: *B D A C*

Similarly, the preorder and postorder traversals output the nodes in the following order:

Preorder sequence: *A B D C*

Postorder sequence: *D B C A*

As you can see from the walk-through of the inorder traversal, after visiting the left subtree of a node we must come back to the node itself. The links are only in one direction; that is, the parent node points to the left and right children, but there is no pointer from each child to the parent. Therefore, before going to a child, we must somehow save a pointer to the parent node. A convenient way to do this is to write a recursive inorder function because in a recursive call after completing a particular call, the control goes back to the caller. (Later we discuss how to write nonrecursive traversal functions.) The recursive definition of the function to implement the inorder traversal algorithms is as follows:

```
template <class elemType>
void inorder(binaryTreeNode<elemType> *p) const
{
    if (p != NULL)
    {
        inorder(p->llink);
        cout << p->info << " ";
        inorder(p->rlink);
    }
}
```

To do the inorder traversal of a binary tree, the root node of the binary tree is passed as a parameter to the function inorder. For example, if the root points to the root node of the binary tree, a call to the function inorder is as follows:

```
inorder(root);
```

Similarly, we can write the functions to implement the preorder and postorder traversals. The definitions of these functions are given next.

```
template <class elemType>
void preorder(binaryTreeNode<elemType> *p) const
{
    if (p != NULL)
    {
        cout << p->info << " ";
        preorder(p->llink);
        preorder(p->rlink);
    }
}

template <class elemType>
void postorder(binaryTreeNode<elemType> *p) const
{
    if (p != NULL)
    {
        postorder(p->llink);
        postorder(p->rlink);
        cout << p->info << " ";
    }
}
```

Implementing Binary Trees

The previous sections described various operations that can be performed on a binary tree, as well as the functions to implement these operations. This section describes binary trees as an abstract data type (ADT). Before designing the class to implement a binary tree as an ADT, let us list various operations that are typically performed on a binary tree:

- Determine whether the binary tree is empty.
- Search the binary tree for a particular item.
- Insert an item in the binary tree.
- Delete an item from the binary tree.
- Find the height of the binary tree.
- Find the number of nodes in the binary tree.
- Find the number of leaves in the binary tree.
- Traverse the binary tree.
- Copy the binary tree.

The item search, insertion, and deletion operations all require the binary tree to be traversed. However, because the nodes of a binary tree are in no particular order, these algorithms are not very efficient on arbitrary binary trees. That is, no criteria exist to guide the search on these binary trees, as we will see in the next section. Therefore, we will discuss these algorithms when we discuss special binary trees.

Other than for the search, insertion, and deletion operations, the following class defines binary trees as an ADT. The definition of the node is the same as before. However, for the sake of completeness and easy reference, we give the definition of the node followed by the definition of the class.

```
//*************************************************************
// Author: D.S. Malik
//
// class binaryTreeType
// This class specifies the basic operations to implement a
// binary tree.
//*************************************************************

     //Definition of the node
template <class elemType>
struct binaryTreeNode
{
    elemType info;
    binaryTreeNode<elemType> *llink;
    binaryTreeNode<elemType> *rlink;
};

   //Definition of the class
template <class elemType>
class binaryTreeType
```

```
{
public:
    const binaryTreeType<elemType>& operator=
                (const binaryTreeType<elemType>&);
      //Overload the assignment operator.
    bool isEmpty() const;
      //Returns true if the binary tree is empty;
      //otherwise, returns false.
    void inorderTraversal() const;
      //Function to do an inorder traversal of the binary tree.
    void preorderTraversal() const;
      //Function to do a preorder traversal of the binary tree.
    void postorderTraversal() const;
      //Function to do a postorder traversal of the binary tree.

    int treeHeight() const;
      //Returns the height of the binary tree.
    int treeNodeCount() const;
      //Returns the number of nodes in the binary tree.
    int treeLeavesCount() const;
      //Returns the number of leaves in the binary tree.
    void destroyTree();
      //Deallocates the memory space occupied by the binary tree.
      //Postcondition: root = NULL;

    binaryTreeType(const binaryTreeType<elemType>& otherTree);
      //copy constructor

    binaryTreeType();
      //default constructor

    ~binaryTreeType();
      //destructor

protected:
    binaryTreeNode<elemType>  *root;

private:
    void copyTree(binaryTreeNode<elemType>* &copiedTreeRoot,
                  binaryTreeNode<elemType>* otherTreeRoot);
      //Makes a copy of the binary tree to which
      //otherTreeRoot points. The pointer copiedTreeRoot
      //points to the root of the copied binary tree.

    void destroy(binaryTreeNode<elemType>* &p);
      //Function to destroy the binary tree to which p points.
      //Postcondition: p = NULL

    void inorder(binaryTreeNode<elemType> *p) const;
      //Function to do an inorder traversal of the binary
      //tree to which p points.
```

```
    void preorder(binaryTreeNode<elemType> *p) const;
      //Function to do a preorder traversal of the binary
      //tree to which p points.
    void postorder(binaryTreeNode<elemType> *p) const;
      //Function to do a postorder traversal of the binary
      //tree to which p points.

    int height(binaryTreeNode<elemType> *p) const;
      //Function to return the height of the binary tree
      //to which p points.
    int max(int x, int y) const;
      //Returns the larger of x and y.
    int nodeCount(binaryTreeNode<elemType> *p) const;
      //Function to return the number of nodes in the binary
      //tree to which p points
    int leavesCount(binaryTreeNode<elemType> *p) const;
      //Function to return the number of leaves in the binary
      //tree to which p points
};
```

Notice that the definition of the `class binaryTreeType` contains the statement to overload the assignment operator, copy constructor, and destructor. This is because the `class binaryTreeType` contains pointer data members. Recall that for classes with pointer data members, the three things that we must do are explicitly overload the assignment operator, include the copy constructor, and include the destructor.

The definition of the `class binaryTreeType` contains several member functions that are `private` members of the class. These functions are used to implement the `public` member functions of the class and the user need not know their existence. For example, to do an inorder traversal, the function `inorderTraversal` calls the function `inorder` and passes the pointer `root` as a parameter to this function. Suppose that you have the following statement:

```
binaryTreeType<int> myTree;
```

The following statement does an inorder traversal of `myTree`:

```
myTree.inorder();
```

Also, note that in the definition of the `class binaryTreeType`, the pointer `root` is declared as a `protected` member so that we can later derive special binary trees.

Next, we give the definitions of the member functions of the `class binaryTreeType`.

The binary tree is empty if `root` is `NULL`. So the definition of the function `isEmpty` is as follows:

```
template <class elemType>
bool binaryTreeType<elemType>::isEmpty() const
{
    return (root == NULL);
}
```

The default constructor initializes the binary tree to an empty state; that is, it sets the pointer `root` to `NULL`. Therefore, the definition of the default constructor is as follows:

```
template <class elemType>
binaryTreeType<elemType>::binaryTreeType()
{
    root = NULL;
}
```

The definitions of the other functions are as follows:

```
template <class elemType>
void binaryTreeType<elemType>::inorderTraversal() const
{
    inorder(root);
}

template <class elemType>
void binaryTreeType<elemType>::preorderTraversal() const
{
    preorder(root);
}

template <class elemType>
void binaryTreeType<elemType>::postorderTraversal() const
{
    postorder(root);
}

template <class elemType>
int binaryTreeType<elemType>::treeHeight() const
{
    return height(root);
}

template <class elemType>
int binaryTreeType<elemType>::treeNodeCount() const
{
    return nodeCount(root);
}

template <class elemType>
int binaryTreeType<elemType>::treeLeavesCount() const
{
    return leavesCount(root);
}

template <class elemType>
void binaryTreeType<elemType>::
                     inorder(binaryTreeNode<elemType> *p) const
```

```
{
    if (p != NULL)
    {
        inorder(p->llink);
        cout << p->info << " ";
        inorder(p->rlink);
    }
}

template <class elemType>
void binaryTreeType<elemType>::
                     preorder(binaryTreeNode<elemType> *p) const
{
    if (p != NULL)
    {
        cout << p->info << " ";
        preorder(p->llink);
        preorder(p->rlink);
    }
}

template <class elemType>
void binaryTreeType<elemType>::
                     postorder(binaryTreeNode<elemType> *p) const
{
    if (p != NULL)
    {
        postorder(p->llink);
        postorder(p->rlink);
        cout << p->info << " ";
    }
}

template <class elemType>
int binaryTreeType<elemType>::
                      height(binaryTreeNode<elemType> *p) const
{
    if (p == NULL)
        return 0;
    else
        return 1 + max(height(p->llink), height(p->rlink));
}

template <class elemType>
int binaryTreeType<elemType>::max(int x, int y) const
{
    if (x >= y)
        return x;
    else
        return y;
}
```

The definitions of the functions `nodeCount` and `leavesCount` are left as exercises for you. See Programming Exercises 1 and 2 at the end of this chapter.

Next, we give the definitions of the functions `copyTree`, `destroy`, and `destroyTree`, as well as the definitions of the copy constructor and the destructor, and overload the assignment operator.

The definition of the function `copyTree` is the same as before; here this function is a member of the `class binaryTreeType`:

```
template <class elemType>
void binaryTreeType<elemType>::copyTree
                      (binaryTreeNode<elemType>* &copiedTreeRoot,
                       binaryTreeNode<elemType>* otherTreeRoot)
{
    if (otherTreeRoot == NULL)
        copiedTreeRoot = NULL;
    else
    {
        copiedTreeRoot = new binaryTreeNode<elemType>;
        copiedTreeRoot->info = otherTreeRoot->info;
        copyTree(copiedTreeRoot->llink, otherTreeRoot->llink);
        copyTree(copiedTreeRoot->rlink, otherTreeRoot->rlink);
    }
} //end copyTree
```

To destroy a binary tree, for each node, first we destroy its left subtree, then its right subtree, and then the node itself. We must use the operator `delete` to deallocate the memory occupied by each node. The definition of the function `destroy` is as follows:

```
template <class elemType>
void binaryTreeType<elemType>::destroy(binaryTreeNode<elemType>* &p)
{
    if (p != NULL)
    {
        destroy(p->llink);
        destroy(p->rlink);
        delete p;
        p = NULL;
    }
}
```

To implement the function `destroyTree`, we use the function `destroy` and pass the root node of the binary tree to the function `destroy`. The definition of the function `destroyTree` is as follows:

```
template <class elemType>
void binaryTreeType<elemType>::destroyTree()
{
    destroy(root);
}
```

Recall that when a class object is passed by value, the copy constructor copies the value of the actual parameters into the formal parameters. Because the `class binaryTreeType` has pointer data members, which creates dynamic memory, we must provide the definition

of the copy constructor to avoid the shallow copying of data. The definition of the copy constructor, given next, uses the function `copyTree` to make an identical copy of the binary tree that is passed as a parameter.

```
    //copy constructor
template <class elemType>
binaryTreeType<elemType>::binaryTreeType
                (const binaryTreeType<elemType>& otherTree)
{
    if (otherTree.root == NULL) //otherTree is empty
        root = NULL;
    else
        copyTree(root, otherTree.root);
}
```

The definition of the destructor is quite straightforward. When an object of type `binaryTreeType` goes out of scope, the destructor deallocates the memory occupied by the nodes of the binary tree. The definition of the destructor uses the function `destroy` to accomplish this task.

```
    //destructor
template <class elemType>
binaryTreeType<elemType>::~binaryTreeType()
{
    destroy(root);
}
```

Next, we discuss the function to overload the assignment operator. To assign the value of one binary tree to another binary tree, we make an identical copy of the binary tree to be assigned by using the function `copyTree`. The definition of the function to overload the assignment operator is as follows:

```
     //overloading the assignment operator
template <class elemType>
const binaryTreeType<elemType>& binaryTreeType<elemType>::operator=
                          (const binaryTreeType<elemType>& otherTree)
{
    if (this != &otherTree) //avoid self-copy
    {
        if (root != NULL)  //if the binary tree is not empty,
                           //destroy the binary tree
            destroy(root);

        if (otherTree.root == NULL) //otherTree is empty
            root = NULL;
        else
            copyTree(root, otherTree.root);
    }//end else

    return *this;
}
```

Binary Search Trees

Now that you know the basic operations on a binary tree, this section discusses a special type of binary tree, called a binary search tree.

Consider the binary tree in Figure 11-6.

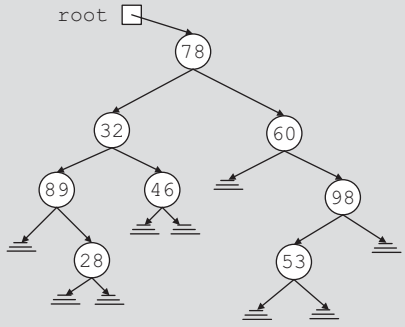


FIGURE 11-6 Arbitrary binary tree

Suppose that we want to determine whether `50` is in the binary tree. To do so, we can use any of the previous traversal algorithms to visit each node and compare the search item with the data stored in the node. However, this could require us to traverse a large part of the binary tree, so the search would be slow. We need to visit each node in the binary tree until either the item is found or we have traversed the entire binary tree because no criteria exist to guide our search. This case is like an arbitrary linked list where we must start our search at the first node, and continue looking at each node until either the item is found or the entire list is searched.

On the other hand, consider the binary tree in Figure 11-7.

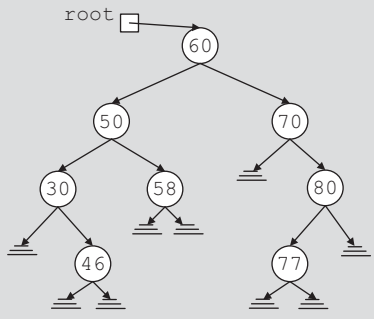


FIGURE 11-7 Binary search tree

In the binary tree in Figure 11-7, the data in each node is

- Larger than the data in its left subtree
- Smaller than the data in its right subtree

The binary tree in Figure 11-7 has some structure. Suppose that we want to determine whether `58` is in this binary tree. As before, we must start our search at the root node. We compare `58` with the data in the root node; that is, we compare `58` with `60`. Because `58` $\neq$ `60` and `58 < 60`, it is guaranteed that `58` will not be in the right subtree of the root node. Therefore, if `58` is in the binary tree, it must be in the left subtree of the root node. We follow the left pointer of the root node and go to the node with `info 50`. We now apply the same criteria at this node. Because `58 > 50`, we must follow the right pointer of this node and go to the node with `info 58`. At this node we find item `58`.

This example shows that every time we move down to a child, we eliminate one of the subtrees of the node from our search. If the binary tree is nicely constructed, the search is very similar to the binary search on arrays.

The binary tree given in Figure 11-7 is a special type of binary tree, called a binary search tree. (In the following definition, by the term *key of the node* we mean the key of the data item that uniquely identifies the item.)

Definition: A binary search tree, T, is either empty or the following is true:

i. T has a special node called the **root** node.
ii. T has two sets of nodes, L_T and R_T, called the left subtree and right subtree of T, respectively.
iii. The key in the root node is larger than every key in the left subtree and smaller than every key in the right subtree.
iv. L_T and R_T are binary search trees.

The following operations are typically performed on a binary search tree:

- Search the binary search tree for a particular item.
- Insert an item in the binary search tree.
- Delete an item from the binary search tree.
- Find the height of the binary search tree.
- Find the number of nodes in the binary search tree.
- Find the number of leaves in the binary search tree.
- Traverse the binary search tree.
- Copy the binary search tree.

Clearly, every binary search tree is a binary tree. The height of a binary search tree is determined the same way as the height of a binary tree. Similarly, the operations to find the number of nodes, to find the number of leaves, and to do inorder, preorder, and postorder traversals of a binary search tree are the same as those for a binary tree. Therefore, we can

inherit all of these operations from the binary tree. That is, we can extend the definition of the binary tree by using the principle of inheritance and, hence, define the binary search tree.

The following class defines a binary search tree as an ADT by extending the definition of the binary tree:

```
//*************************************************************
// Author: D.S. Malik
//
// This class specifies the basic operations to implement a
// binary search tree.
//*************************************************************

template <class elemType>
class bSearchTreeType: public binaryTreeType<elemType>
{
public:
    bool search(const elemType& searchItem) const;
      //Function to determine if searchItem is in the binary
      //search tree.
      //Postcondition: Returns true if searchItem is found in the
      //    binary search tree; otherwise, returns false.

    void insert(const elemType& insertItem);
      //Function to insert insertItem in the binary search tree.
      //Postcondition: If no node in the binary search tree has the
      //    same info as insertItem, a node with the info insertItem
      //    is created and inserted in the binary search tree.

    void deleteNode(const elemType& deleteItem);
      //Function to delete deleteItem from the binary search tree.
      //Postcondition: If a node with the same info as deleteItem
      //    is found, it is deleted from the binary search tree.

private:
    void deleteFromTree(binaryTreeNode<elemType>* &p);
      //Function to delete the node to which p points is deleted
      //from the binary search tree.
      //Postcondition: The node to which p points is deleted from
      //    the binary search tree.
};
```

Next, we describe each of these operations.

Search

The function `search` searches the binary search tree for a given item. If the item is found in the binary search tree, it returns `true`; otherwise, it returns `false`. Because the pointer `root` points to the root node of the binary search tree, we must begin our search at the root node. Furthermore, because `root` must always point to the root node, we need a pointer, say `current`, to traverse the binary search tree. The pointer `current` is initialized to `root`.

If the binary search tree is nonempty, we first compare the search item with the info in the root node. If they are the same, we stop the search and return `true`. Otherwise, if the search item is smaller than the info in the node, we follow `llink` to go to the left subtree; otherwise, we follow `rlink` to go to the right subtree. We repeat this process for the next node. If the search item is in the binary search tree, our search ends at the node containing the search item; otherwise, the search ends at an empty subtree. Thus, the general algorithm is as follows:

```
if root is NULL
    Cannot search an empty tree, returns false.
else
{
    current = root;
    while (current is not NULL and not found)
        if (current->info is the same as the search item)
            set found to true;
        else if(current->info is greater than the search item)
            follow the llink of current
        else
            follow the rlink of current
}
```

This pseudocode algorithm translates into the following C++ function:

```
template <class elemType>
bool bSearchTreeType<elemType>::
                        search(const elemType& searchItem) const
{
    binaryTreeNode<elemType> *current;
    bool found = false;

    if (root == NULL)
       cerr << "Cannot search the empty tree." << endl;
    else
    {
        current = root;

        while (current != NULL && !found)
        {
            if (current->info == searchItem)
                found = true;
            else if (current->info > searchItem)
                current = current->llink;
            else
                current = current->rlink;
        }//end while
    }//end else

    return found;
}//end search
```

Insert

After inserting an item in a binary search tree, the resulting binary tree must also be a binary search tree. To insert a new item, first we search the binary search tree and find the place where the new item is to be inserted. The search algorithm is similar to the search algorithm of the function `search`. Here we traverse the binary search tree with two pointers—a pointer, say `current`, to check the current node and a pointer, say `trailCurrent`, pointing to the parent of current. Because duplicate items are not allowed, our search must end at an empty subtree. We can then use the pointer `trailCurrent` to insert the new item at the proper place. The item to be inserted, `insertItem`, is passed as a parameter to the function `insert`. The general algorithm is as follows:

a. Create a new node and copy `insertItem` into the new node. Also set `llink` and `rlink` of the new node to `NULL`.

b. `if` the `root` is `NULL`, the tree is empty so make `root` point to the new node.

```
else
{
    current = root;
    while (current is not NULL)    //search the binary tree
    {
        trailCurrent = current;
        if (current->info is the same as the insertItem)
            Error: Cannot insert duplicate
            exit
        else if(current->info > insertItem)
            Follow llink of current
        else
            Follow rlink of current
    }

    //insert the new node in the binary tree

    if (trailCurrent->info > insertItem)
        make the new node the left child of trailCurrent
    else
        make the new node the right child of trailCurrent
}
```

This pseudocode algorithm translates into the following C++ function:

```
template <class elemType>
void bSearchTreeType<elemType>::insert(const elemType& insertItem)
{
    binaryTreeNode<elemType> *current;  //pointer to traverse the tree
    binaryTreeNode<elemType> *trailCurrent; //pointer behind current
    binaryTreeNode<elemType> *newNode;  //pointer to create the node
```

```
    newNode = new binaryTreeNode<elemType>;
    assert(newNode != NULL);
    newNode->info = insertItem;
    newNode->llink = NULL;
    newNode->rlink = NULL;

    if (root == NULL)
        root = newNode;
    else
    {
        current = root;

        while (current != NULL)
        {
            trailCurrent = current;

            if (current->info == insertItem)
            {
                cerr << "The insert item is already in the list-";
                cerr << "duplicates are not allowed."
                     << insertItem << endl;
                return;
            }
            else if (current->info > insertItem)
                current = current->llink;
            else
                current = current->rlink;
        }//end while

        if (trailCurrent->info > insertItem)
            trailCurrent->llink = newNode;
        else
            trailCurrent->rlink = newNode;
    }
}//end insert
```

Delete

As before, first we search the binary search tree to find the node to be deleted. To help you better understand the delete operation, before describing the function to delete an item from the binary search tree, let us consider the binary search tree given in Figure 11-8.

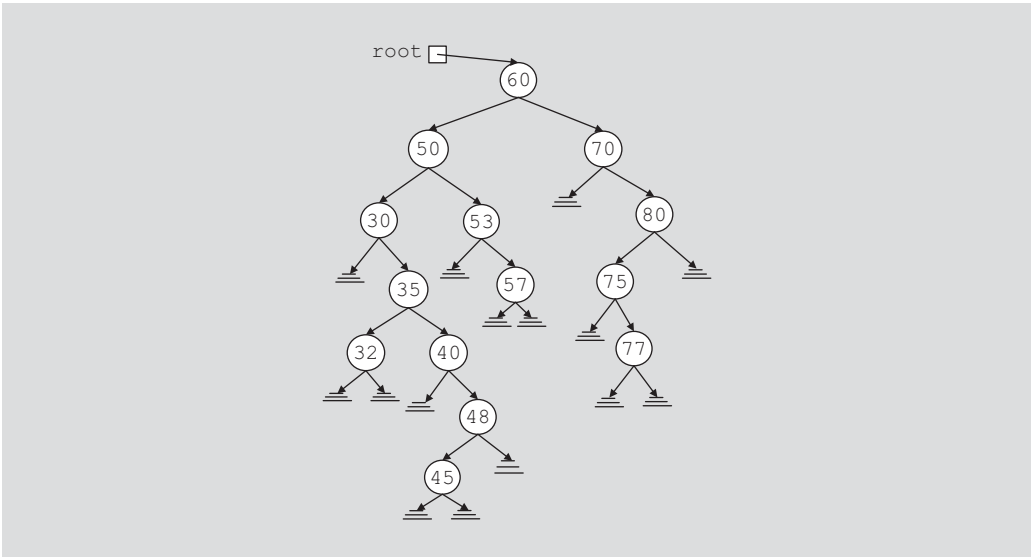


FIGURE 11-8 Binary search tree before deleting a node

After deleting the desired item (if it exists in the binary search tree), the resulting tree must be a binary search tree. The delete operation has four cases, as follows:

Case 1: The node to be deleted has no left and right subtrees; that is, the node to be deleted is a leaf. For example, the node with `info 45` is a leaf.

Case 2: The node to be deleted has no left subtree; that is, the left subtree is empty, but it has a nonempty right subtree. For example, the left subtree of node with `info 40` is empty and its right subtree is nonempty.

Case 3: The node to be deleted has no right subtree; that is, the right subtree is empty, but it has a nonempty left subtree. For example, the left subtree of node with `info 80` is empty and its right subtree is nonempty.

Case 4: The node to be deleted has nonempty left and right subtrees. For example, the left and the right subtrees of node with `info 50` are nonempty.

Figure 11-9 illustrates these four cases.

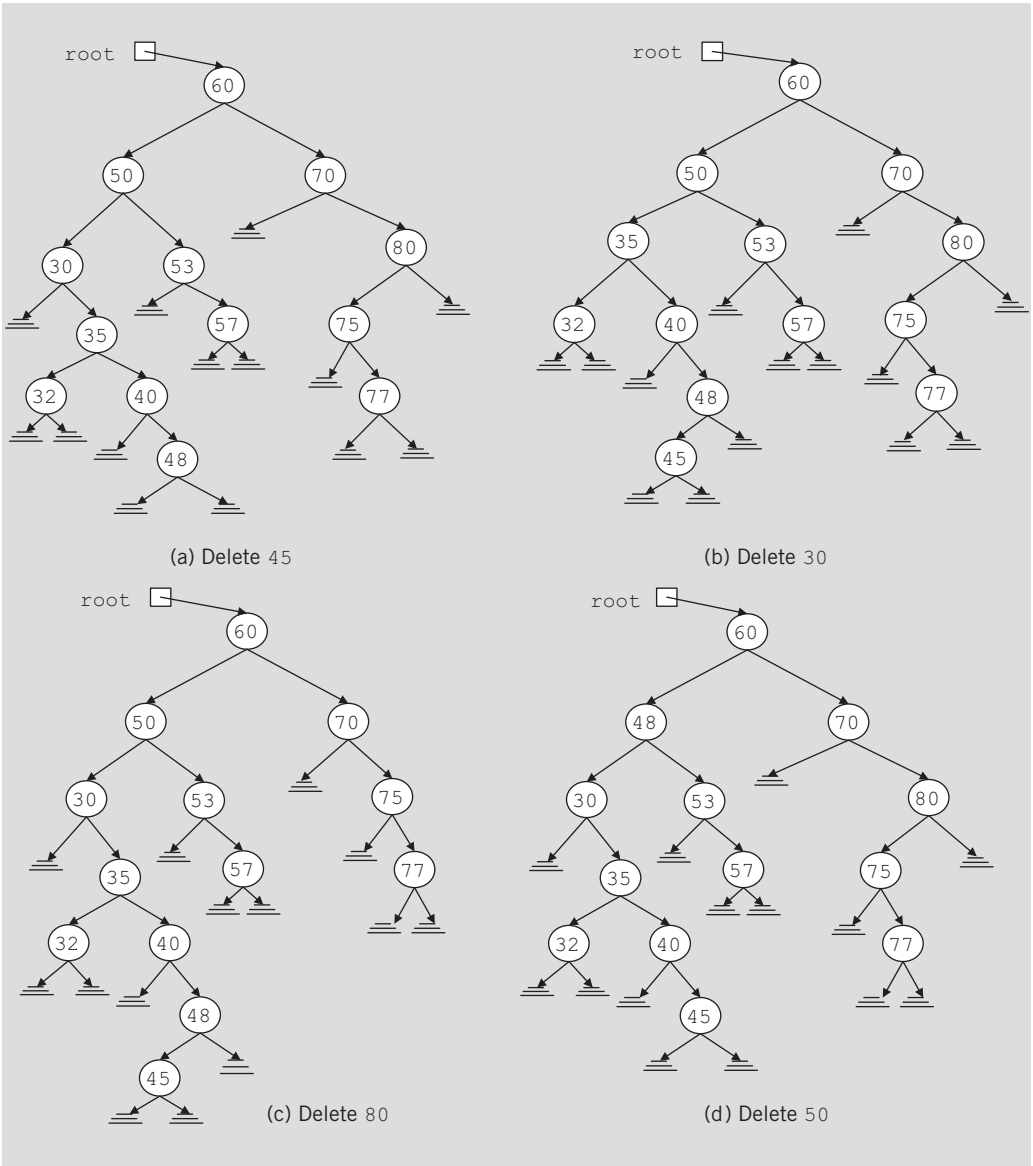


FIGURE 11-9 The binary tree of Figure 11-8 after deleting various items

Case 1: Suppose that we want to delete `45` from the binary search tree in Figure 11-8. We search the binary tree and arrive at the node containing `45`. Because this node is a leaf and is the left child of its parent, we can simply set the `llink` of the parent node to `NULL` and deallocate the memory occupied by this node. After deleting this node, Figure 11-9(a) shows the resulting binary search tree.

Case 2: Suppose that we want to delete 30 from the binary search tree in Figure 11-8. In this case, the node to be deleted has no left subtree. Because 30 is the left child of its parent node, we make the `llink` of the parent node point to the right child of 30 and then deallocate the memory occupied by 30. Figure 11-9(b) shows the resulting binary tree.

Case 3: Suppose that we want to delete 80 from the binary search tree of Figure 11-8. The node containing 80 has no right child and is the right child of its parent. Thus, we make the `rlink` of the parent of 80—that is, 70—point to the left child of 80. Figure 11-9(c) shows the resulting binary tree.

Case 4: Suppose that we want to delete 50 from the binary search tree in Figure 11-8. The node with `info` 50 has a nonempty left subtree and a nonempty right subtree. Here, we first reduce this case to either Case 2 or Case 3 as follows. To be specific, suppose that we reduce it to Case 3—that is, the node to be deleted has no right subtree. For this case, we find the immediate predecessor of 50 in this binary tree, which is 48. This is done by first going to the left child of 50 and then locating the rightmost node of the left subtree of 50. To do so, we follow the `rlink` of the nodes. Because the binary search tree is finite, we eventually arrive at a node that has no right subtree. Next, we swap the info in the node to be deleted with the info of its immediate predecessor. In this case, we swap 48 with 50. This reduces to the case wherein the node to be deleted has no right subtree. We now apply Case 3 to delete the node. (Note that because we will delete the immediate predecessor from the binary tree, we, in fact, copy only the info of the immediate predecessor into the node to be deleted.) After deleting 50 from the binary search tree in Figure 11-8, the resulting binary tree is as shown in Figure 11-9(d).

In each case, we see that the resulting binary tree is again a binary search tree.

From this discussion, it follows that to delete an item from a binary search tree, we must do the following:

1. Find the node containing the item (if any) to be deleted.
2. Delete the node.

We accomplish the second step by a separate function, which we will call `deleteFromTree`. Given a pointer to the node to be deleted, this function deletes the node by taking into account the previous four cases.

The preceding examples show that whenever we delete a node from a binary tree, we adjust one of the pointers of the parent node. Because the adjustment has to be made in the parent node, we must call the function `deleteFromTree` by using an appropriate pointer of the parent node. For example, suppose that the node to be deleted is 35, which is the right child of its parent node. Further suppose that `trailCurrent` points to the node containing 30, the parent node of 35. A call to the function `deleteFromTree` is as follows:

```
deleteFromTree(trailCurrent->rlink);
```

Of course, if the node to be deleted is the root node, then the call to the function `deleteFromTree` is as follows:

```
deleteFromTree(root);
```

We now define the C++ function `deleteFromTree`:

```
template <class elemType>
void bSearchTreeType<elemType>::deleteFromTree
                                (binaryTreeNode<elemType>* &p)
{
    binaryTreeNode<elemType> *current;//pointer to traverse the tree
    binaryTreeNode<elemType> *trailCurrent; //pointer behind current
    binaryTreeNode<elemType> *temp;    //pointer to delete the node

    if (p == NULL)
        cerr << "Error: The node to be deleted is NULL." << endl;
    else if(p->llink == NULL && p->rlink == NULL)
    {
        temp = p;
        p = NULL;
        delete temp;
    }
    else if(p->llink == NULL)
    {
        temp = p;
        p = temp->rlink;
        delete temp;
    }
    else if(p->rlink == NULL)
    {
        temp = p;
        p = temp->llink;
        delete temp;
    }
    else
    {
        current = p->llink;
        trailCurrent = NULL;

        while (current->rlink != NULL)
        {
            trailCurrent = current;
            current = current->rlink;
        }//end while

        p->info = current->info;

        if (trailCurrent == NULL) //current did not move;
                                  //current == p->llink; adjust p
            p->llink = current->llink;
        else
            trailCurrent->rlink = current->llink;

        delete current;
    }//end else
}//end deleteFromTree
```

Next, we describe the function `deleteNode`. The function `deleteNode` first searches the binary search tree to find the node containing the item to be deleted. The item to be deleted, `deleteItem`, is passed as a parameter to the function. If the node containing `deleteItem` is found in the binary search tree, the function `deleteNode` calls the function `deleteFromTree` to delete the node. The definition of the function `deleteNode` is given next.

```
template <class elemType>
void bSearchTreeType<elemType>::deleteNode(const elemType& deleteItem)
{
    binaryTreeNode<elemType> *current; //pointer to traverse the tree
    binaryTreeNode<elemType> *trailCurrent; //pointer behind current
    bool found = false;

    if (root == NULL)
        cout << "Cannot delete from the empty tree." << endl;
    else
    {
        current = root;
        trailCurrent = root;

        while (current != NULL && !found)
        {
            if (current->info == deleteItem)
                found = true;
            else
            {
                trailCurrent = current;

                if (current->info > deleteItem)
                    current = current->llink;
                else
                    current = current->rlink;
            }
        }//end while

        if (current == NULL)
            cout << "The delete item is not in the tree." << endl;
        else if (found)
        {
            if (current == root)
                deleteFromTree(root);
            else if (trailCurrent->info > deleteItem)
                deleteFromTree(trailCurrent->llink);
            else
                deleteFromTree(trailCurrent->rlink);
        }//end if
    }
}//end deleteNode
```

Binary Search Tree: Analysis

This section provides an analysis of the performance of binary search trees. Let T be a binary search tree with n nodes, where $n > 0$. Suppose that we want to determine whether an item, x, is in T. The performance of the search algorithm depends on the shape of T. Let us first consider the worst case. In the worst case, T is linear. That is, T is one of the forms shown in Figure 11-10.

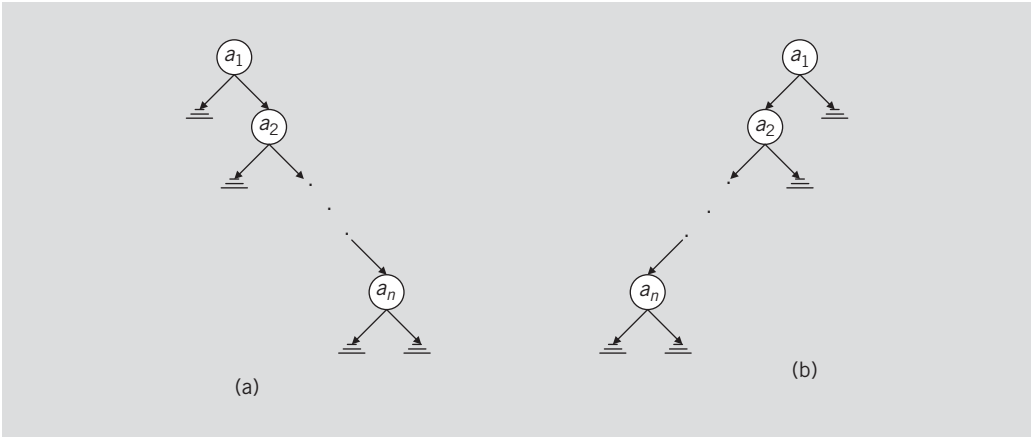


FIGURE 11-10 Linear binary trees

Because T is linear, the performance of the search algorithm on T is the same as its performance on a linear list. Therefore, in the successful case, on average, the search algorithm makes $(n + 1) / 2$ key comparisons. In the unsuccessful case, it makes n comparisons.

Let us now consider the average-case behavior. In the successful case, the search would end at a node. Because there are n items, there are $n!$ possible orderings of the keys. We assume that all $n!$ orderings of the keys are possible. Let $S(n)$ denote the number of comparisons in the average successful case, and $U(n)$ denote the number of comparisons in the average unsuccessful case.

The number of comparisons required to determine whether x is in T is one more than the number of comparisons required to insert x in T. Furthermore, the number of comparisons required to insert x in T is the same as the number of comparisons made in the unsuccessful search, reflecting that x is not in T. From this, it follows that

$$S(n) = 1 + \frac{U(0) + U(1) + \ldots + U(n-1)}{n} \qquad \text{(Equation 11-1)}$$

It is also known that

$$S(n) = \left(1 + \frac{1}{n}\right) U(n) - 3 \qquad \text{(Equation 11-2)}$$

Solving Equations (11-1) and (11-2), it can be shown that $U(n) \approx 2.77\log_2 n$ and $S(n) \approx 1.39\log_2 n$.

We can now formulate the following result.

Theorem: Let T be a binary search tree with n nodes, where $n > 0$. The average number of nodes visited in a search of T is approximately $1.39\log_2 n = O(\log_2 n)$ and the number of key comparisons is approximately $2.77 \log_2 n = O(\log_2 n)$.

Nonrecursive Binary Tree Traversal Algorithms

The previous sections described how to do the following:

- Traverse a binary tree using the inorder, preorder, and postorder methods.
- Construct a binary tree.
- Insert an item in a binary tree.
- Delete an item from a binary tree.

The traversal algorithms—inorder, preorder, and postorder—discussed earlier are recursive. Because traversing a binary tree is a fundamental operation and recursive functions are somewhat less efficient then their iterative versions, this section discusses the nonrecursive inorder, preorder, and postorder traversal algorithms.

Nonrecursive Inorder Traversal

In the inorder traversal of a binary tree, for each node, the left subtree is visited first, then the node, and then the right subtree. It follows that in an inorder traversal, the first node visited is the leftmost node of the binary tree. For example, in the binary tree in Figure 11-11, the leftmost node is the node with `info 28`.

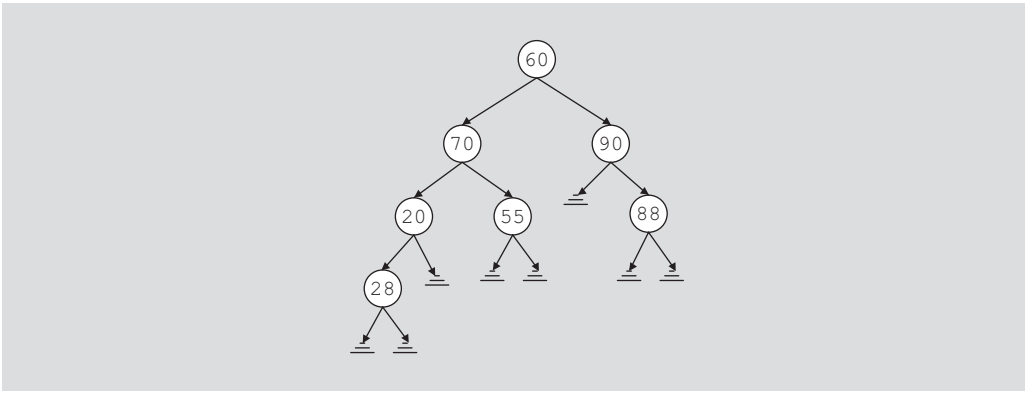


FIGURE 11-11 Binary tree; leftmost node is 28

To get to the leftmost node of the binary tree, we start by traversing the binary tree at the root node and then follow the left link of each node until the left link of a node becomes null. We then back up to the parent node, visit the node, and then move to the right node. Because links go in only one direction, to get back to a node, we must save a pointer to the node before moving to the child node. Moreover, the nodes must be backtracked in the order they were traversed. It follows that while backtracking, the nodes must be visited in a last-in, first-out manner. This can be done by using a stack. We, therefore, save a pointer to a node in a stack. The general algorithm is as follows:

1. `current = root; //start traversing the binary tree at the root node`
2. `while` (`current` is not `NULL` or `stack` is nonempty)
   `if` (`current` is not `NULL`)
   `{`
   push `current` into the `stack`;
   `current = current->llink;`
   `}`
   `else`
   `{`
   pop `stack` into `current`;
   visit `current;` `//visit the node`
   `current = current->rlink;` `//move to the right child`
   `}`

The following function implements the nonrecursive inorder traversal of a binary tree:

```
template <class elemType>
void binaryTreeType<elemType>::nonRecursiveInTraversal() const
{
    stackType<binaryTreeNode<elemType>* > stack;
    binaryTreeNode<elemType> *current;
    current = root;

    while ((current != NULL) || (!stack.isEmptyStack()))
        if (current != NULL)
        {
            stack.push(current);
            current = current->llink;
        }
        else
        {
            current = stack.top();
            stack.pop();
            cout << current->info << " ";
            current = current->rlink;
        }

    cout << endl;
}
```

Nonrecursive Preorder Traversal

In a preorder traversal of a binary tree, for each node, first the node is visited, then the left subtree is visited, and then the right subtree is visited. As in the case of an inorder traversal, after visiting a node and before moving to the left subtree, we must save a pointer to the node so that after visiting the left subtree, we can visit the right subtree. The general algorithm is as follows:

1. `current = root;` `//start the traversal at the root node`
2. `while` (`current` is not `NULL` or `stack` is nonempty)
   `if` (`current` is not `NULL`)
   {
   visit `current` node;
   `push current` into `stack`;
   `current = current->llink`;
   }
   `else`
   {
   `pop stack` into `current`;
   `current = current->rlink; //prepare to visit the`
   `//right subtree`
   }

The following function implements the nonrecursive preorder traversal algorithm:

```
template <class elemType>
void binaryTreeType<elemType>::nonRecursivePreTraversal() const
{
    stackType<binaryTreeNode<elemType>*> stack;
    binaryTreeNode<elemType> *current;

    current = root;

    while ((current != NULL) || (!stack.isEmptyStack()))
        if (current != NULL)
        {
            cout << current->info << " ";
            stack.push(current);
            current = current->llink;
        }
        else
        {
            current = stack.top();
            stack.pop();
            current = current->rlink;
        }

    cout << endl;
}
```

Nonrecursive Postorder Traversal

In a postorder traversal of a binary tree, for each node, first the left subtree is visited, then the right subtree is visited, and then the node is visited. As in the case of an inorder traversal, in a postorder traversal, the first node visited is the leftmost node of the binary tree. Because—for each node—the left and right subtrees are visited before visiting the node, we must indicate to the node whether the left and right subtrees have been visited. After visiting the left subtree of a node and before visiting the node, we must visit its right subtree. Therefore, after returning from a left subtree, we must tell the node that the right subtree needs to be visited, and after visiting the right subtree we must tell the node that it can now be visited. To do this, other than saving a pointer to the node (to get back to the right subtree and to the node itself), we also save an integer value of 1 before moving to the left subtree and an integer value of 2 before moving to the right subtree. Whenever the stack is popped, the integer value associated with that pointer is popped as well. This integer value tells whether the left and right subtrees of a node have been visited.

The general algorithm is as follows:

1. `current = root; //start the traversal at the root node`
2. `v = 0;`
3. `if` (`current` is `NULL`)
   the binary tree is empty
4. `if` (`current` is not `NULL`)
   a. `push current` into `stack;`
   b. `push 1` into `stack;`
   c. `current = current->llink;`
   d. `while` (`stack` is not empty)

```
if (current is not NULL and v is 0)
{
    push current and 1 into stack;
    current = current->llink;
}
else
{
    pop stack into current and v;
    if (v == 1)
    {
        push current and 2 into stack;
        current = current->rlink;
        v = 0;
    }
    else
        visit current;
}
```

We use two (parallel) stacks: one to save a pointer to a node and another to save the integer value (1 or 2) associated with this pointer. We leave it as an exercise for you write

the definition of a C++ function to implement the preceding postorder traversal algorithm; see Programming Exercise 6 at the end of this chapter.

Binary Tree Traversal and Functions as Parameters

Suppose that you have stored employee data in a binary search tree, and at the end of the year pay increases or bonuses are to be awarded to each employee. This task requires that each node in the binary search tree be visited and that the salary of each employee be updated. The preceding sections discussed various ways to traverse a binary tree. However, in these traversal algorithms—inorder, preorder, and postorder—whenever we visited a node, for simplicity and for illustration purposes, we only output the data contained in each node. How do we use a traversal algorithm to visit each node and update the data in each node? One way to do so is to first create another binary search tree in which the data in each node is the updated data of the original binary search tree, and then destroy the old binary search tree. This would require extra computer time and perhaps extra memory, and, therefore, is not efficient. Another solution is to write separate traversal algorithms to update the data. This solution requires you to frequently modify the definition of the class implementing the binary search tree. However, if the user can write an appropriate function to update the data of each employee and then pass this function as a parameter to the traversal algorithms, we can considerably enhance the program's flexibility. This section describes how to pass functions as parameters to other functions.

In C++, a function name without any parentheses is considered a pointer to the function. To specify a function as a formal parameter to another function, we specify the function type, followed by the function name as a pointer, followed by the parameter types of the function. For example, consider the following statements:

```
void fParamFunc1(void (*visit) (int));         //Line 1
void fParamFunc2(void (*visit) (elemType&));  //Line 2
```

The statement in Line 1 declares `fParamFunc1` to be a function that takes as a parameter any `void` function that has one value parameter of type `int`. The statement in Line 2 declares `fParamFunc2` to be a function that takes as a parameter any `void` function that has one reference parameter of type `elemType`.

We can now rewrite the inorder traversal function of the `class binaryTreeType`. Alternatively, we can overload the existing inorder traversal functions. To further illustrate function overloading, we will overload the inorder traversal functions. Therefore, we include the following statements in the definition of the `class binaryTreeType`:

```
void inorderTraversal(void (*visit) (elemType&));
  //Function to do an inorder traversal of the binary tree.
  //The parameter visit, which is a function, specifies the
  //action to be taken at each node.
```

```
void inorder(binaryTreeNode<elemType> *p, void (*visit) (elemType&));
  //Function to do an inorder traversal of the binary
  //tree, starting at the node specified by the parameter p.
  //The parameter visit, which is a function, specifies the
  //action to be taken at each node.
```

The definitions of these functions are as follows:

```
template <class elemType>
void binaryTreeType<elemType>::inorderTraversal
                               (void (*visit) (elemType& item))
{
    inorder(root, *visit);
}

template <class elemType>
void binaryTreeType<elemType>::inorder(binaryTreeNode<elemType>* p,
                               void (*visit) (elemType& item))
{
    if (p != NULL)
    {
        inorder(p->llink, *visit);
        (*visit)(p->info);
        inorder(p->rlink, *visit);
    }
}
```

The statement

```
(*visit)(p->info);
```

in the definition of the function `inorder` makes a call to the function with one reference parameter of the type `elemType` pointed to by the pointer `visit`.

Example 11-3 further illustrates how functions are passed as parameters to other functions.

EXAMPLE 11-3

This example shows how to pass a user-defined function as a parameter to the binary tree traversal algorithms. For illustration purposes, we show how to use only the inorder traversal function.

The following program uses the `class bSearchTreeType`, which is derived from the `class binaryTreeType`, to build the binary tree. The traversal functions are included in the `class binaryTreeType`, which are then inherited by the `class bSearchTreeType`.

```
//*************************************************************
// Author: D.S. Malik
//
// This program illustrates how to pass a user-defined function
// as a parameter to the binary tree traversal algorithms.
//*************************************************************
```

```
#include <iostream>                                          //Line 1
#include "binarySearchTree.h"                                //Line 2

using namespace std;                                         //Line 3

void print(int& x);                                          //Line 4
void update(int& x);                                         //Line 5

int main()                                                   //Line 6
{                                                            //Line 7
    bSearchTreeType<int>  treeRoot;                          //Line 8

    int num;                                                 //Line 9

    cout << "Line 10: Enter numbers ending with -999"
         << endl;                                            //Line 10
    cin >> num;                                              //Line 11

    while (num != -999)                                      //Line 12
    {                                                        //Line 13
        treeRoot.insert(num);                                //Line 14
        cin >> num;                                          //Line 15
    }                                                        //Line 16

    cout << endl << "Line 17: Tree nodes in inorder: ";      //Line 17
    treeRoot.inorderTraversal(print);                        //Line 18
    cout << endl << "Line 19: Tree Height: "
         << treeRoot.treeHeight()
         << endl << endl;                                    //Line 19

    cout << "Line 20: ******* Update Nodes *******"
         << endl;                                            //Line 20
    treeRoot.inorderTraversal(update);                       //Line 21

    cout << "Line 22: Tree nodes in inorder after "
         << "the update: " << endl << "          ";         //Line 22
    treeRoot.inorderTraversal(print);                        //Line 23
    cout << endl <<"Line 24: Tree Height: "
         << treeRoot.treeHeight() << endl;                   //Line 24

    return 0;                                                //Line 25
}                                                            //Line 26

void print(int& x)                                           //Line 27
{                                                            //Line 28
    cout << x << " ";                                        //Line 29
}                                                            //Line 30

void update(int& x)                                          //Line 31
{                                                            //Line 32
    x = 2 * x;                                               //Line 33
}                                                            //Line 34
```

Sample Run: In this sample run, the user input is shaded.

```
Line 10: Enter numbers ending with -999
56 87 23 65 34 45 12 90 66 -999

Line 17: Tree nodes in inorder: 12 23 34 45 56 65 66 87 90
Line 19: Tree Height: 4

Line 20: ******* Update Notes *******
Line 22: Tree nodes in inorder after the update:
         24 46 68 90 112 130 132 174 180
Line 24: Tree Height: 4
```

This program works as follows. The statement in Line 8 declares `treeRoot` to be a binary search tree object, in which the data in each node is of type `int`. The statements in Lines 11 through 16 build the binary search tree. The statement in Line 18 uses the member function `inorderTraversal` of `treeRoot` to traverse the binary search tree `treeRoot`. The parameter to the function `inorderTraversal`, in Line 18, is the function `print` (defined at Line 27). Because the function `print` outputs the value of its argument, the statement in Line 18 outputs the data of the nodes of the binary search tree `treeNode`. The statement in Line 19 outputs the height of the binary search tree.

The statement in Line 21 uses the member function `inorderTraversal` to traverse the binary search tree `treeRoot`. In Line 21, the actual parameter of the function `inorderTraversal` is the function `update` (defined at Line 31). The function `update` doubles the value of its argument. Therefore, the statement in Line 21 updates the data of each node of the binary search tree by doubling the value. The statements in Lines 23 and 24 output the nodes and the height of the binary search tree.

AVL (Height-Balanced) Trees

In the previous sections, you learned how to build and manipulate a binary search tree. The performance of the search algorithm on a binary search tree depends on how the binary tree is built. The shape of the binary search tree depends on the data set. If the data set is sorted, the binary search tree is linear and so the search algorithm would not be efficient. On the other hand, if the tree is nicely built, the search would be fast. In fact, the smaller the height of the tree, the faster the search. Therefore, we want the height of the binary search tree to be as small as possible. This section describes a special type of binary search tree, called the **AVL tree** (also called the **height-balanced tree**) in which the resulting binary search is nearly balanced. AVL trees are due to the mathematicians G. M. Adelson-Velskii and E. M. Landis and are so named in their honor. The methods of building such binary trees were given by them in 1962.

We begin by defining the following terms.

Definition: A **perfectly balanced** binary tree is a binary tree such that

i. The heights of the left and right subtrees of the root are equal.
ii. The left and right subtrees of the root are perfectly balanced binary trees.

Figure 11-12 shows a perfectly balanced binary tree.

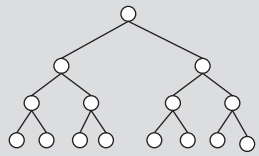


FIGURE 11-12 Perfectly balanced binary tree

Let T be a binary tree and x be a node in T. If T is perfectly balanced, then from the definition of the perfectly balanced tree, it follows that the height of the left subtree of x is the same as the height of the right subtree of x.

It can be proved that, if T is a perfectly balanced binary tree of height h, then the number of nodes in T is $2^h - 1$. From this, it follows that if the number of items in the data set is not equal to $2^h - 1$ for some nonnegative integer h, then we cannot construct a perfectly balanced binary tree. Moreover, perfectly balanced binary trees are a too stringent refinement.

Definition: An **AVL tree** (or **height-balanced tree**) is a binary search tree such that

i. The heights of the left and right subtrees of the root differ by at most 1.
ii. The left and right subtrees of the root are AVL trees.

Figure 11-13 gives examples of AVL and non-AVL trees.

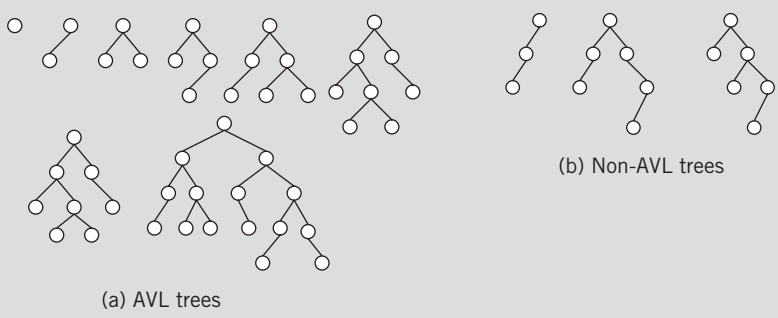


FIGURE 11-13 AVL and non-AVL trees

Let x be a node in a binary tree. Let x_l denote the height of the left subtree of x, and x_h denote the height of the right subtree of x.

Proposition: Let T be an AVL tree and x be a node in T. Then $|x_h - x_l| \leq 1$, where $|x_h - x_l|$ denotes the absolute value of $x_h - x_l$.

Let x be a node in the AVL tree T.

1. If $x_l > x_h$, we say that x is **left high**. In this case, $x_l = x_h + 1$.
2. If $x_l = x_h$, we say that x is **equal high**.
3. If $x_h > x_l$, we say that x is **right high**. In this case, $x_h = x_l + 1$.

Definition: The **balance factor** of x, written $bf(x)$, is defined by $bf(x) = x_h - x_l$.

Let x be a node in the AVL tree T. Then,

1. If x is left high, $bf(x) = -1$.
2. If x is equal high, $bf(x) = 0$.
3. If x is right high, $bf(x) = 1$.

Definition: Let x be a node in a binary tree. We say that the node x **violates the balance criteria** if $|x_h - x_l| > 1$, that is, the heights of the left and right subtrees of x differ by more than 1.

From the preceding discussion, it follows that in addition to the data and pointers to the left and right subtrees, one more thing is associated with each node x in the AVL tree T, which is the balance factor of x. Thus, every node must keep track of its balance factor. To make the algorithms efficient, we store the balance factor of each node in the node itself. Hence, the definition of a node in the AVL tree is as follows:

```
template<class elemType>
struct AVLNode
{
    elemType info;
    int bfactor;  //balance factor
    AVLNode<elemType> *llink;
    AVLNode<elemType> *rlink;
};
```

Because an AVL tree is a binary search tree, the search algorithm for an AVL tree is the same as the search algorithm for a binary search tree. Other operations, such as finding the height, determining the number of nodes, checking whether the tree is empty, tree traversal, and so on, on AVL trees can be implemented exactly the same way they are implemented on binary trees. However, item insertion and deletion operations on AVL trees are somewhat different from the ones discussed for binary search trees. This is because after inserting (or deleting) a node from an AVL tree, the resulting binary tree must be an AVL tree. Next, we describe these operations.

Insertion

To insert an item in an AVL tree, first we search the tree and find the place where the new item is to be inserted. Because an AVL tree is a binary search tree, to find the place

for the new item we can search the AVL tree using a search algorithm similar to the search algorithm designed for binary search trees. If the item to be inserted is already in the tree, the search ends at a nonempty subtree. Because duplicates are not allowed, in this case we can output an appropriate error message. Suppose that the item to be inserted is not in the AVL tree. Then, the search ends at an empty subtree and we insert the item in that subtree. After inserting the new item in the tree, the resulting tree might not be an AVL tree. Thus, we must restore the tree's balance criteria. This is accomplished by traveling the same path, back to the root node, which was followed when the new item was inserted in the AVL tree. The nodes on this path (back to the root node) are visited and either their balance factor is changed, or we might have to reconstruct part of the tree. We illustrate these cases with the help of the following examples.

NOTE In Figures 11-14 to 11-17, for each node, we show only the data stored in the node. Furthermore, an equal sign, `=`, on the top of a node indicates that the balance factor of this node is `0`, the less-than sign, `<`, indicates that the balance factor of this node is `-1`, and the greater-than sign, `>`, indicates that the balance factor of this node is `1`.

Consider the AVL tree of Figure 11-14 (a). Let us insert `90` into this AVL tree.

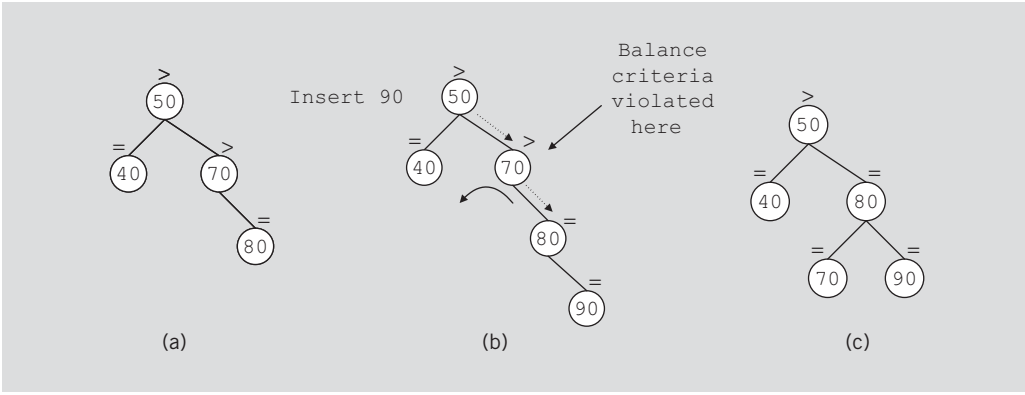


FIGURE 11-14 AVL tree before and after inserting 90

We search the tree starting at the root node to find the place for `90`. The dotted arrow shows the path. We insert a node with `info 90` and obtain the tree of Figure 11-14(b). The binary tree of Figure 11-14(b) is not an AVL tree. So we backtrack and go to node `80`. Prior to insertion, `bf(80)` was `0`. Because the new node was inserted into the (empty) right subtree of `80`, we change its balance factor to 1 (not shown in the figure). Now we go back to node `70`. Prior to insertion, `bf(70)` was `1`. Because after insertion the height of the right subtree of `70` is increased, we see that the subtree with root node

70 is not an AVL tree. In this case, we reconstruct this subtree (this is called rotating the tree at root node 70). Therefore, we obtain the tree shown in Figure 11-14(c). The binary tree of Figure 11-14 is an AVL tree.

Now consider the AVL tree of Figure 11-15(a). Let us insert 75 into the AVL tree of Figure 11-15(a).

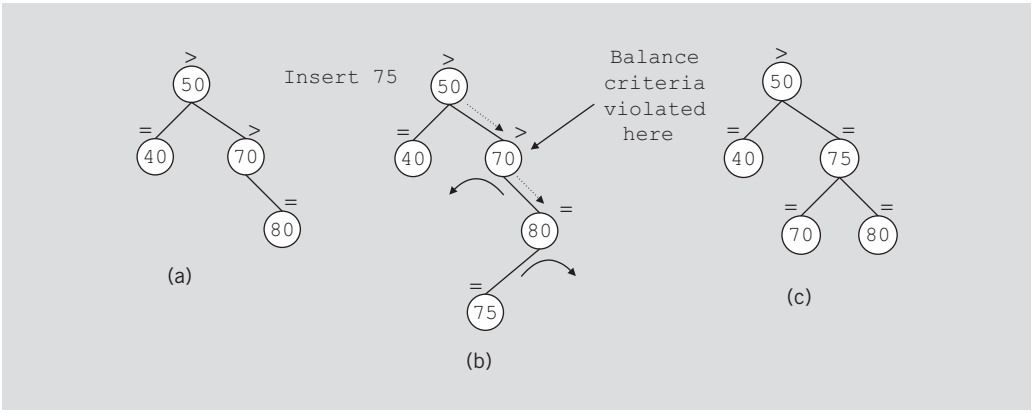


FIGURE 11-15 AVL tree before and after inserting 75

As before, we search the tree starting at the root node. The dotted arrows show the path traversed. After inserting 75, the resulting tree is shown in Figure 11-15(b). After inserting 75, we backtrack. First we go to node 80 and change its balance factor to -1. The subtree with root node 80 is an AVL tree. Now we go back to 70. Clearly, the subtree with root node 70 is not an AVL tree. So we construct this subtree. In this case, we first reconstruct the subtree at root node 80, and then reconstruct the subtree at root node 70 to obtain the tree shown in Figure 11-15(c). (These constructions, that is, rotations, are explained in the next section, "AVL Tree Rotations.")

Notice that in Figures 11-14(c) and 11-15(c), after reconstructing the subtree at the node, the subtree no longer grew in height. At this point, we usually send the message that overall the tree did not gain any height to the remaining nodes on the path back to the root node of the tree, and so the remaining nodes on the path do not need to do anything.

Next, consider the AVL tree of Figure 11-16. Let us insert `95` into this AVL tree.

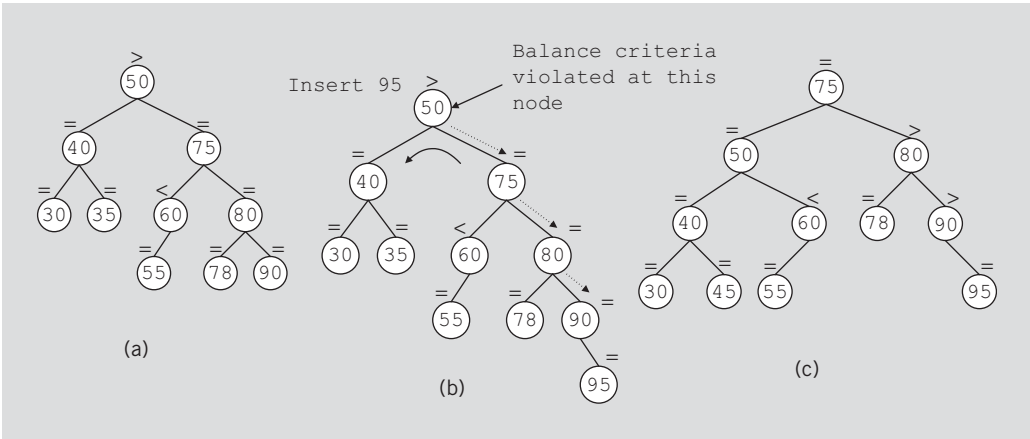


FIGURE 11-16 AVL tree before and after inserting 95

We search the tree and insert `95`, as shown in Figure 11-16(b). After inserting `95`, we see that the subtrees with root nodes `90`, `80`, and `75` are still AVL trees. When backtracking the path, we simply adjust the balance factor of these nodes (if needed). However, when we backtrack to the root node, we discover that the tree at this node is no longer an AVL tree. Prior to insertion, `bf(50)` was `1`, that is, its right subtree was higher than its left subtree. After insertion, the subtree grew in height, thus violating the balance criteria at `50`. So we construct the tree at node `50`. In this case, the tree will be reconstructed as shown in Figure 11-16(c).

Before discussing the general algorithms for reconstructing (rotating) a subtree, let us consider one more case. Consider the AVL tree as shown in Figure 11-17(a). Let us insert `88` in this tree.

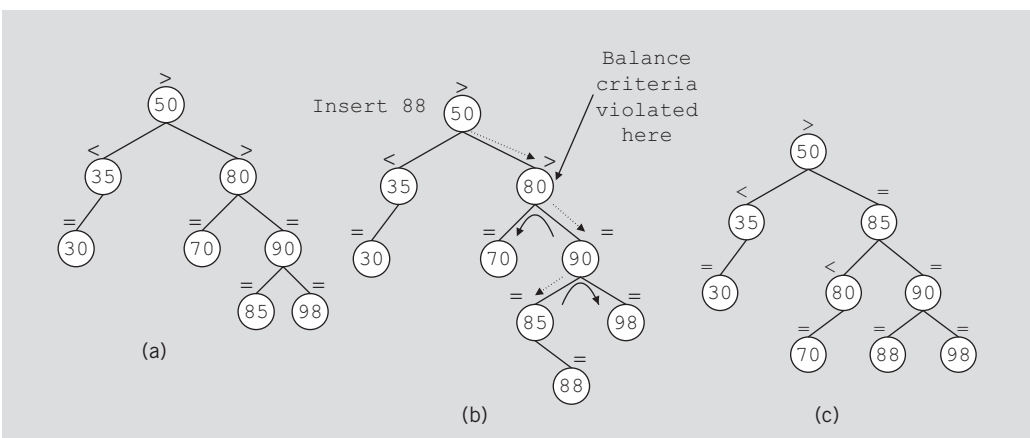


FIGURE 11-17 AVL tree before and after inserting 88

Following the insertion procedure as described previously, we obtain the tree as shown in Figure 11-17(b). As before, we now backtrack to the root node. We adjust the balance factor of nodes `85` and `90`. When we visit node `80`, we discover that at this node we need to reconstruct the subtree. In this case, the subtree is reconstructed as shown in Figure 11-17(c). As before, after reconstructing the subtree, the entire tree is balanced. So for the remaining nodes on the path back to the root node, we would not do anything.

The examples described previously indicate that if part of the tree requires reconstruction, then after reconstructing that part of the tree, we can ignore the remaining nodes on the path back to the root node. (This is, indeed, the case.) Also, after inserting the node, the reconstruction can occur at any node on the path back to the root node.

AVL Tree Rotations

We now describe the reconstruction procedure, called **rotating** the tree. There are two types of rotations: **left rotation** and **right rotation**. Suppose the rotation occurs at a node *x*. If it is a left rotation, then certain nodes from the right subtree of *x* move to its left subtree; the root of the right subtree of *x* becomes the new root of the reconstructed subtree. Similarly, if it is a right rotation at *x*, certain nodes from the left subtree of *x* move to its right subtree; the root of the left subtree of *x* becomes the new root of the reconstructed subtree.

Case 1: Consider Figure 11-18.

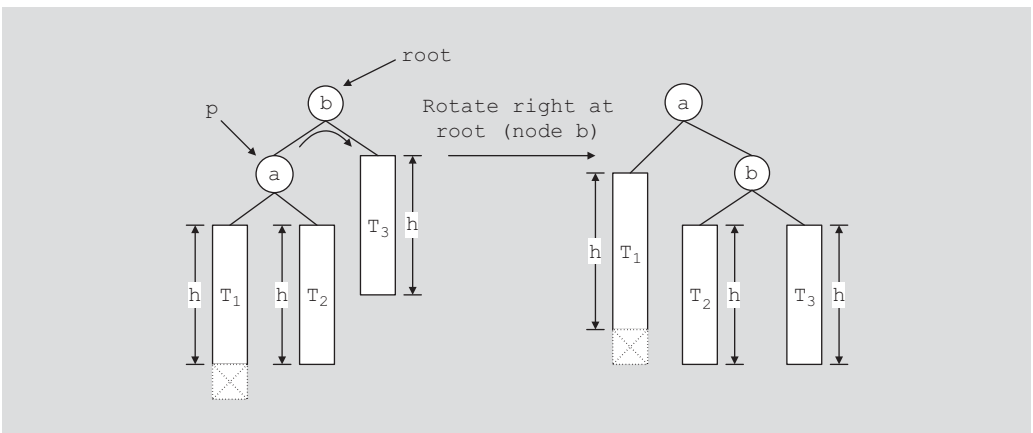


FIGURE 11-18 Right rotation at *b*

In Figure 11-18, subtrees T_1, T_2, and T_3 are of equal height, say *h*. The dotted rectangle shows an item insertion in T_1, causing the height of the subtree T_1 to increase by 1. The subtree at node *a* is still an AVL tree, but the balance criteria is violated at the root node. We note the following in this tree. Because the tree is a binary search tree,

- Every key in subtree T_1 is smaller than the key in node *a*.
- Every key in subtree T_2 is larger than the key in node *a*.
- Every key in subtree T_2 is smaller than the key in node *b*.

Therefore,

1. We make T_2 (the right subtree of node *a*) the left subtree of node *b*.
2. We make node *b* the right child of node *a*.
3. Node *a* becomes the root of the reconstructed tree, as shown in Figure 11-18.

Case 2: This case is a mirror image of Case 1. See Figure 11-19.

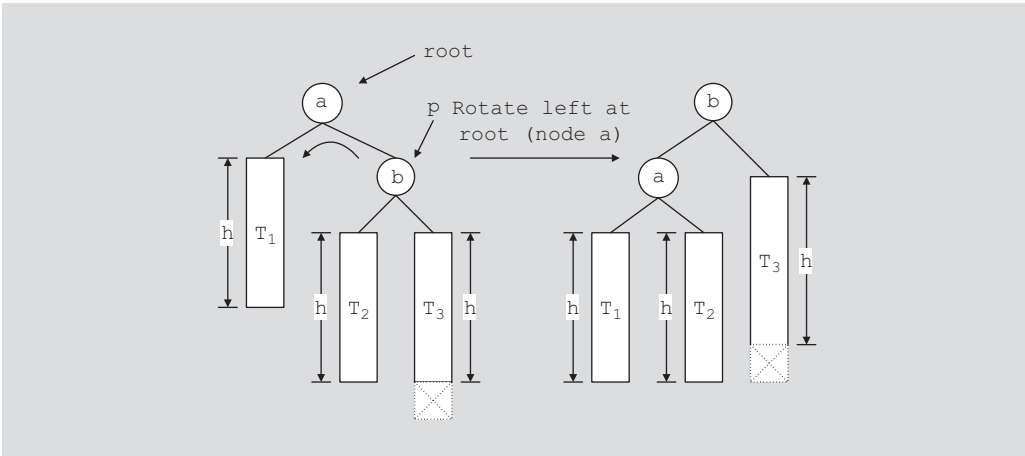


FIGURE 11-19 Left rotation at *a*

Case 3: Consider Figure 11-20.

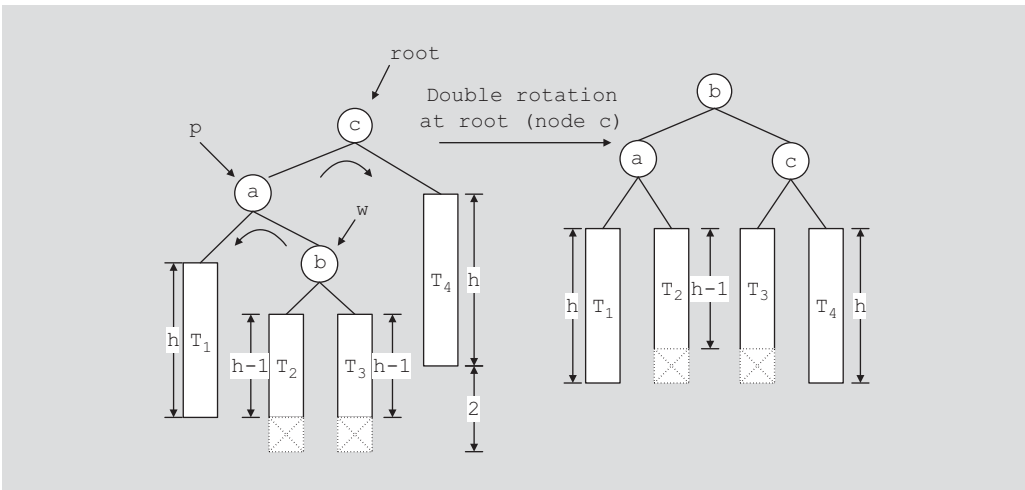


FIGURE 11-20 Double rotation: First rotate left at *a* and then right at *c*

In Figure 11-20, the tree on the left is the tree prior to the reconstruction. The heights of the subtrees are shown in the figure. The dotted rectangle shows that a new item is inserted in the subtree, causing the subtree to grow in height. The new item is inserted either in T_2 or T_3. We note the following (in the tree prior to reconstruction):

- All keys in T_3 are smaller than the key in node *c*.
- All keys in T_3 are larger than the key in node *b*.
- All keys in T_2 are smaller than the key in node *b*.
- All keys in T_2 are larger than the key in node *a*.
- After insertion, the subtrees with root nodes *a* and *b* are still AVL trees.
- The balance criteria is violated at the root node, *c*, of the tree.
- The balance factors of node *c*, $bf(c) = -1$, and node *a*, $bf(a) = 1$ are opposite.

This is an example of double rotation. One rotation is required at node *a*, and another rotation is required at node *c*. If the balance factor of the node, where the tree is to be reconstructed, and the balance factor of the higher subtree are opposite, that node requires a double rotation. First, we rotate the tree at node *a* and then at node *c*. Now the tree at node *a* is right high and so we make a left rotation at *a*. Next, because the tree at node *c* is left high, we make a right rotation at *c*. Figure 11-20 shows the resulting tree (which is to the right of the tree after insertion). Figure 11-21, however, shows both rotations in sequence.

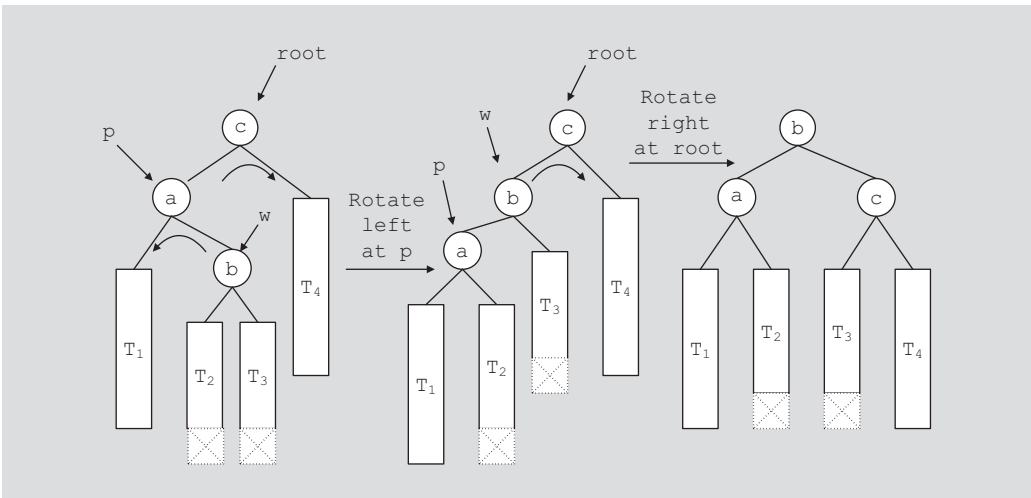


FIGURE 11-21 Left rotation at *a* followed by a right rotation at *c*

Case 4: This is a mirror image of Case 3. We illustrate this with the help of Figure 11-22.

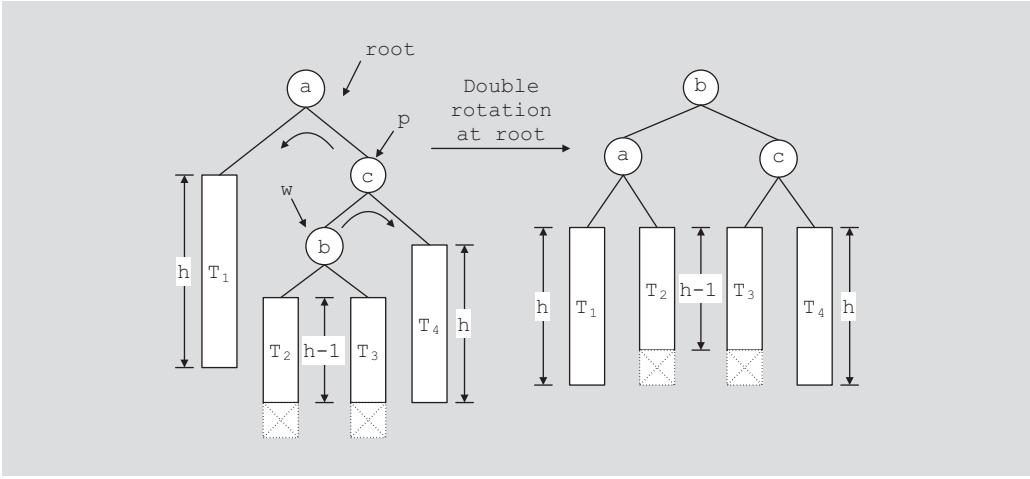


FIGURE 11-22 Double rotation: First rotate right at *c*, then rotate left at *a*

Using these four cases, we now describe what type of rotation might be required at a node.

Suppose that the tree is to be reconstructed, by rotation, at node x. Then, the subtree with root node x requires either a single or a double rotation.

1. Suppose that the balance factor of the node x and the balance factor of the root node of the higher subtree of x have the same sign, that is, both positive or both negative.
   a. If these balance factors are positive, make a single *left* rotation at x. (Prior to insertion, the right subtree of x was higher than its left subtree. The new item was inserted in the right subtree of x, causing the height of the right subtree to increase in height, which in turn violated the balance criteria at x.)
   b. If these balance factors are negative, make a single *right* rotation at x. (Prior to insertion, the left right subtree of x was higher than its right subtree. The new item was inserted in the left subtree of x, causing the height of the left subtree to increase in height, which in turn violated the balance criteria at x.)
2. Suppose that the balance factor of the node x and the balance factor of the higher subtree of x are opposite in sign. To be specific, suppose that the balance factor of node x prior to insertion was -1 and suppose that y is the root node of the left subtree of x. After insertion, the balance factor of node y is 1. That is, after insertion, the right subtree of node y grew in height. In this case, we require a *double* rotation at x. First, we make a left rotation at y (because y is right high). Then, we make a right rotation at x. The other case, which is a mirror image of this case, is handled similarly.

The following C++ functions implement the left and right rotations of a node. The pointer of the node requiring the rotation is passed as a parameter to the function.

```
template <class elemT>
void rotateToLeft(AVLNode<elemT>* &root)
{
    AVLNode<elemT> *p;  //pointer to the root of the
                        //right subtree of root
    if (root == NULL)
        cerr << "Error in the tree" << endl;
    else if (root->rlink == NULL)
        cerr << "Error in the tree:"
             <<" No right subtree to rotate." << endl;
    else
    {
        p = root->rlink;
        root->rlink = p->llink; //the left subtree of p becomes
                                //the right subtree of root
        p->llink = root;
        root = p;   //make p the new root node
    }
}//rotateLeft

template <class elemT>
void rotateToRight(AVLNode<elemT>* &root)
{
    AVLNode<elemT> *p;  //pointer to the root of
                        //the left subtree of root

    if (root == NULL)
        cerr << "Error in the tree" << endl;
    else if (root->llink == NULL)
        cerr << "Error in the tree:"
             << " No left subtree to rotate." << endl;
    else
    {
        p = root->llink;
        root->llink = p->rlink; //the right subtree of p becomes
                                //the left subtree of root
        p->rlink = root;
        root = p;   //make p the new root node
    }
}//end rotateRight
```

Now that we know how to implement both rotations, we next write the C++ functions, `balanceFromLeft` and `balanceFromRight`, which are used to reconstruct the tree at a particular node. The pointer of the node where reconstruction occurs is passed as a parameter to this function. These functions use the functions `rotateToLeft` and `rotateToRight` to reconstruct the tree, and also adjust the balance factors of the nodes affected by the reconstruction. The function `balanceFromLeft` is called when the

subtree is left double high and certain nodes need to be moved to the right subtree. The function `balanceFromRight` has similar conventions.

```
template <class elemT>
void balanceFromLeft(AVLNode<elemT>* &root)
{
    AVLNode<elemT> *p;
    AVLNode<elemT> *w;

    p = root->llink;   //p points to the left subtree of root

    switch (p->bfactor)
    {
    case -1:
        root->bfactor = 0;
        p->bfactor = 0;
        rotateToRight(root);
        break;

    case 0:
        cerr << "Error: Cannot balance from the left." << endl;
        break;

    case 1:
        w = p->rlink;
        switch (w->bfactor)  //adjust the balance factors
        {
        case -1:
            root->bfactor = 1;
            p->bfactor = 0;
            break;

        case 0:
            root->bfactor = 0;
            p->bfactor = 0;
            break;

        case 1:
            root->bfactor = 0;
            p->bfactor = -1;
        }//end switch

        w->bfactor = 0;
        rotateToLeft(p);
        root->llink = p;
        rotateToRight(root);
    }//end switch;
}//end balanceFromLeft
```

For the sake of completeness, we also give the definition of the function `balanceFromRight`:

```
template <class elemT>
void balanceFromRight(AVLNode<elemT>* &root)
{
    AVLNode<elemT> *p;
    AVLNode<elemT> *w;

    p = root->rlink;   //p points to the left subtree of root

    switch (p->bfactor)
    {
    case -1:
        w = p->llink;
        switch (w->bfactor)  //adjust the balance factors
        {
        case -1:
            root->bfactor = 0;
            p->bfactor = 1;
            break;

        case 0:
            root->bfactor = 0;
            p->bfactor = 0;
            break;

        case 1:
            root->bfactor = -1;
            p->bfactor = 0;
        }//end switch

        w->bfactor = 0;
        rotateToRight(p);
        root->rlink = p;
        rotateToLeft(root);
        break;

    case 0:
        cerr << "Error: Cannot balance from the left." << endl;
        break;

    case 1:
        root->bfactor = 0;
        p->bfactor = 0;
        rotateToLeft(root);
    }//end switch;
}//end balanceFromRight
```

We now focus our attention on the function `insertIntoAVL`. The function `insertIntoAVL` inserts the new item in the AVL tree. The item to be inserted and the pointer of the root node of the AVL tree are passed as parameters to this function.

The following steps describe the function `insertIntoAVL`:

1. Create a node and copy the item to be inserted into the newly created node.
2. Search the tree and find the place for the new node in the tree.
3. Insert the new node in the tree.
4. Backtrack the path, which was constructed to find the place for the new node in the tree, to the root node. If necessary, adjust the balance factors of the nodes, or reconstruct the tree at a node on the path.

Because Step 4 requires us to backtrack the path to the root node, and in the binary tree we have links only from the parent to the children, the easiest way to implement the function `insertIntoAVL` is to use recursion. (Recall that recursion automatically takes care of the backtracking.) This is exactly what we do. The function `insertIntoAVL` also uses a reference `bool` parameter, `isTaller`, to indicate to the parent whether the subtree grew in height or not.

```
template <class elemT>
void insertIntoAVL(AVLNode<elemT>* &root,
                   AVLNode<elemT>  *newNode, bool& isTaller)
{
    if (root == NULL)
    {
        root = newNode;
        isTaller = true;
    }
    else if (root->info == newNode->info)
        cerr << "No duplicates are allowed." << endl;
    else if (root->info > newNode->info) //newItem goes in
                                         //the left subtree
    {
        insertIntoAVL(root->llink, newNode, isTaller);

        if (isTaller) //after insertion, the subtree grew in height
            switch (root->bfactor)
            {
            case -1:
                balanceFromLeft(root);
                isTaller = false;
                break;

            case 0:
                root->bfactor = -1;
                isTaller = true;
                break;
```

```
            case 1:
                root->bfactor = 0;
                isTaller = false;
            }//end switch
        }//end if

        else
        {
            insertIntoAVL(root->rlink, newNode, isTaller);

            if (isTaller) //after insertion, the subtree grew in
                          //height
                switch (root->bfactor)
                {
                case -1:
                    root->bfactor = 0;
                    isTaller = false;
                    break;

                case 0:
                    root->bfactor = 1;
                    isTaller = true;
                    break;

                case 1:
                    balanceFromRight(root);
                    isTaller = false;
                }//end switch
        }//end else
}//insertIntoAVL
```

Next, we illustrate the `insertIntoAVL` function and build an AVL tree from scratch. Initially the tree is empty. Each figure shows the item to be inserted as well as the balance factor of each node. An equal sign, `=`, on the top of a node indicates that the balance factor of this node is `0`, the less-than sign, `<`, indicates that the balance factor of this node is `-1`, and the greater-than sign, `>`, indicates that the balance factor of this node is `1`.

Figure 11-23 shows how items are inserted into an initially empty AVL tree.

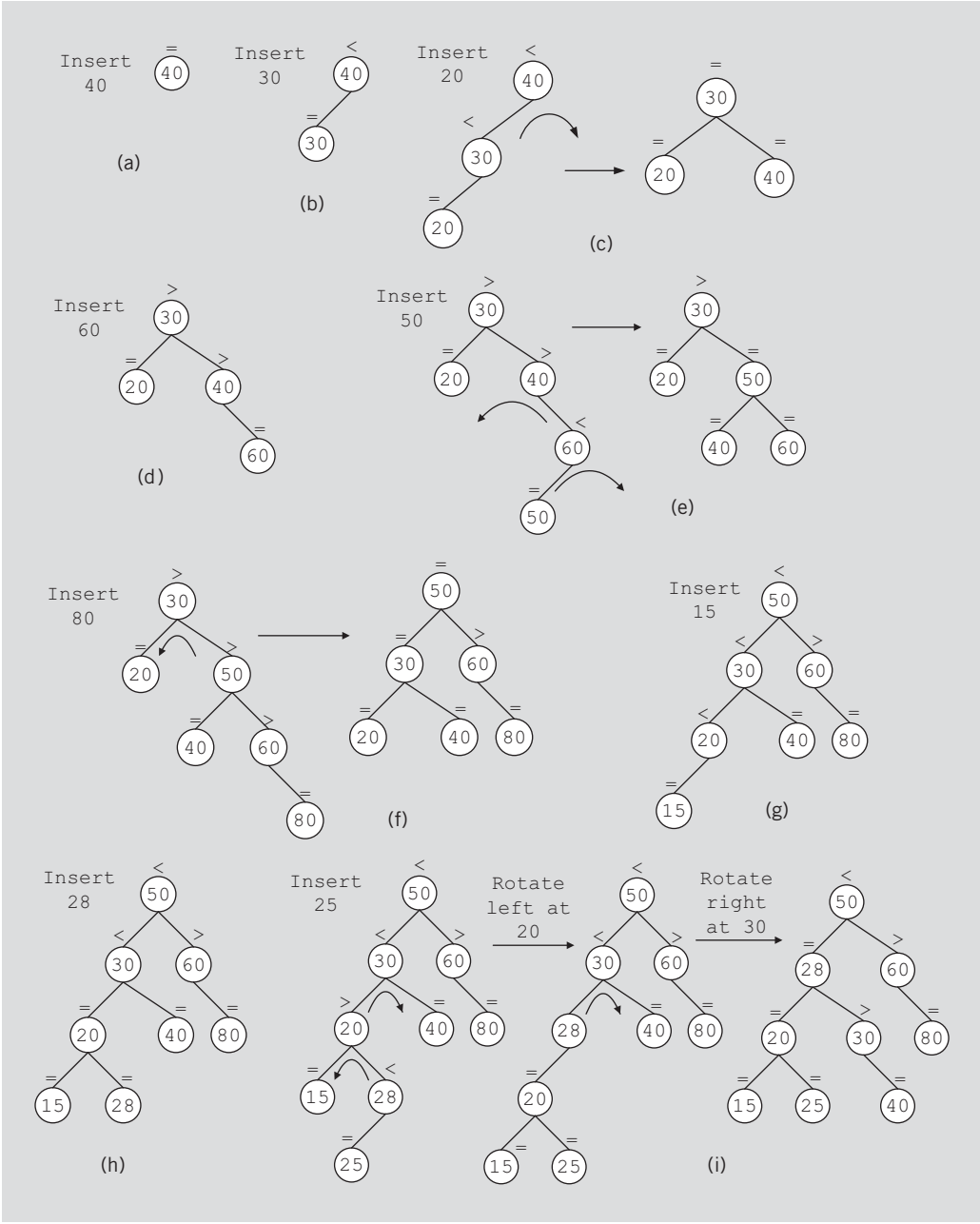


FIGURE 11-23 Item insertion into an initially empty AVL tree

First, we insert 40; see Figure 11-23(a). Next, we insert 30 into the AVL tree. Item 30 is inserted into the left subtree of node 40, causing the left subtree of 40 to grow in height. After insertion, the balance factor of node 40 is -1; see Figure 11-23(b).

Next, we insert 20 into the AVL tree. See Figure 11-23(c). The insertion of 20 violates the balance criteria at node 40. The tree is reconstructed at node 40 by making a single right rotation.

Next, we insert 60 into the AVL tree. See Figure 11-23(d). The insertion of 60 does not require reconstruction; only the balance factor is adjusted at nodes 40 and 30.

Next, we insert 50. See Figure 11-23(e). The insertion of 50 requires the tree to be reconstructed at 40. Notice that a double rotation is made at node 40.

Next, we insert 80; see Figure 11-23(f). The insertion of 80 requires the tree to be reconstructed at node 30.

Next, we insert 15; see Figure 11-23(g). The insertion of node 15 does not require any part of the tree to be reconstructed. We need to only adjust the balance factor of nodes 20, 30, and 50.

Next, we insert 28; see Figure 11-23(h). The insertion of node 28 also does not require any part of the tree to be reconstructed. We need only to adjust the balance factor of node 20.

Next, we insert 25. The insertion of 25 requires a double rotation at node 30. Figure 11-23(i) shows both rotations in sequence. In Figure 11-23(i), the tree is first rotated left at node 20 and then right at node 30.

The following function creates a node, stores the info in the node, and calls the function `insertIntoAVL` to insert the new node in the AVL tree:

```
template <class elemT>
void insert(const elemT &newItem)
{
    bool isTaller = false;
    AVLNode<elemT>  *newNode;

    newNode = new AVLNode<elemT>;
    newNode->info = newItem;
    newNode->bfactor = 0;
    newNode->llink = NULL;
    newNode->rlink = NULL;

    insertIntoAVL(root, newNode, isTaller);
}
```

We leave it as an exercise for you to design the class to implement AVL trees as an ADT. (Notice that because the structure of the node of an AVL tree is different from the structure of the node of a binary tree discussed in the beginning of this chapter, you cannot use inheritance to derive the class to implement AVL trees from the `class binaryTreeType`.)

Deletion from AVL Trees

To delete an item from an AVL tree, first we find the node containing the item to be deleted. The following four cases arise:

Case 1: The node to be deleted is a leaf.

Case 2: The node to be deleted has no right child, that is, its right subtree is empty.

Case 3: The node to be deleted has no left child, that is, its left subtree is empty.

Case 4: The node to be deleted has a left child and a right child.

Cases 1–3 are easier to handle than Case 4. Let us first discuss Case 4.

Suppose that the node to be deleted, say *x*, has a left and a right child. As in the case of deletion from a binary search tree, we reduce Case 4 to Case 2. That is, we find the immediate predecessor, say *y* of *x*. Then, the data of *y* is copied into *x* and now the node to be deleted is *y*. Clearly, *y* has no right child.

To delete the node, we adjust one of the pointers of the parent node. After deleting the node, the resulting tree might no longer be an AVL tree. As in the case of insertion into an AVL tree, we traverse the path (from the parent node) back to the root node. For each node on this path, sometimes we need to change only the balance factor, while other times the tree at a particular node is reconstructed. The following steps describe what to do to a node on the path back to the root node. (As in the case of insertion, we use the `bool` variable `shorter` to indicate whether the height of the subtree is reduced.) Let *p* be a node on the path back to the root node. We look at the current balance factor of *p*.

1. If the current balance factor of *p* is equal high, the balance factor of *p* is changed according to if the left subtree of *p* was shortened or the right subtree of *p* was shortened. The variable `shorter` is set to `false`.
2. Suppose that the balance factor of *p* is not equal and the taller subtree of *p* is shortened. The balance factor of *p* is changed to equal high, and the variable `shorter` is left as `true`.
3. Suppose that the balance factor of *p* is not equal and the shorter subtree of *p* is shortened. Further suppose that *q* points to the root of the taller subtree of *p*.
    a. If the balance factor of *q* is equal, a single rotation is required at *p* and `shorter` is set to `false`.
    b. If the balance factor of *q* is the same as *p*, a single rotation is required at *p* and `shorter` is set to `true`.
    c. Suppose that the balance factors of *p* and *q* are opposite. A double rotation is required at *p* (a single rotation at *q* and then a single rotation at *p*). We adjust the balance factors and set `shorter` to `true`.

Analysis: AVL Trees

Consider all possible AVL trees of height h. Let T_h be an AVL tree of height h such that T_h has the fewest number of nodes. Let T_{hl} denote the left subtree of T_h and T_{hr} denote the right subtree of T_h. Then

$$|T_h| = |T_{hl}| + |T_{hr}| + 1,$$

where $|T_h|$ denotes the number of nodes in T_h.

Because T_h is an AVL tree of height h such that T_h has the fewest number of nodes, it follows that one of the subtrees of T_h is of height $h - 1$ and the other is of height $h - 2$. To be specific, suppose T_{hl} is of height $h - 1$ and T_{hr} is of height $h - 2$. From the definition of T_h, it follows that T_{hl} is an AVL tree of height $h - 1$ such that T_{hl} has the fewest number of nodes among all AVL trees of height $h - 1$. Similarly, T_{hr} is an AVL tree of height $h - 2$ that has the fewest number of nodes among all AVL trees of height $h - 2$. Thus, T_{hl} is of the form T_{h-1} and T_{hr} is of the form T_{h-2}. Hence,

$$|T_h| = |T_{h-1}| + |T_{h-2}| + 1.$$

Clearly,

$$|T_0| = 1$$
$$|T_1| = 2$$

Let $F_{h+2} = |T_h| + 1$. Then,

$$F_{h+2} = F_{h+1} + F_h$$
$$F_2 = 2$$
$$F_3 = 3.$$

This is called a Fibonacci sequence. The solution to F_h is given by

$$F_h \approx \frac{\phi^h}{\sqrt{5}}, \text{ where } \phi = \frac{1+\sqrt{5}}{2}.$$

Hence,

$$|T_h| \approx \frac{\phi^{h+2}}{\sqrt{5}} = \frac{1}{\sqrt{5}}\left[\frac{1+\sqrt{5}}{2}\right]^{h+2}.$$

From this, it can be concluded that

$$h \approx (1.44)\log_2 |T_h|.$$

This implies that, in the worst case, the height of an AVL tree with n nodes is approximately $(1.44)\log_2 n$. Because the height of a perfectly balanced binary tree with n nodes is $\log_2 n$, it follows that, in the worst case, the time to manipulate an AVL tree is no more than 44% of the optimum time. However, in general, AVL trees are not as sparse as in the worst case. It can be shown that the average search time of an AVL tree is about 4% more than the optimum.

PROGRAMMING EXAMPLE: Video Store (Revisited)

In Chapter 5, we designed a program to help a video store to automate its video rental process. That program used an (unordered) linked list to keep track of the video inventory in the store. Because the search algorithm on a linked list is sequential and the list is fairly large, the search could be time consuming. In this chapter, you learned how to organize data into a binary tree. If the binary tree is nicely constructed (that is, it is not linear), the search algorithm can be improved considerably. Moreover, in general, item insertion and deletion in a binary search tree is faster than in a linked list. We will, therefore, redesign the video store program so that the video inventory can be maintained in a binary tree. As in Chapter 5, we leave the design of the customer list in a binary tree as an exercise for you.

VIDEO OBJECT In Chapter 5, a linked list was used to maintain a list of videos in the store. Because the linked list was unordered, to see whether a particular video was in stock, the sequential search algorithm used the equality operator for comparison. However, in the case of a binary tree, we need other relational operators for the search, insertion, and deletion operations. We, therefore, overload all of the relational operators. Other than this difference, the `class videoType` is the same as before. However, we give its definition, without the documentation, here for easy reference and for the sake of completeness.

```
#include <iostream>
#include <string>

using namespace std;

class videoType
{
    friend ostream& operator<<(ostream&, const videoType&);

public:
    void setVideoInfo(string title, string star1,
                      string star2, string producer,
                      string director, string productionCo,
                      int setInStock);
```

```
    int getNoOfCopiesInStock() const;
    void checkOut();
    void checkIn();
    void printTitle() const;
    void printInfo() const;
    bool checkTitle(string title);
    void updateInStock(int num);
    void setCopiesInStock(int num);
    string getTitle();
    videoType(string title = "", string star1 = "",
              string star2 = "", string producer = "",
              string director = "", string productionCo = "",
              int setInStock = 0);

    bool operator==(const videoType&) const;
    bool operator!=(const videoType&) const;
    bool operator<(const videoType&) const;
    bool operator<=(const videoType&) const;
    bool operator>(const videoType&) const;
    bool operator>=(const videoType&) const;

private:
    string videoTitle;
    string movieStar1;
    string movieStar2;
    string movieProducer;
    string movieDirector;
    string movieProductionCo;
    int copiesInStock;
};
```

The definitions of the member functions of the `class videoType` are the same as in Chapter 5. Because here we are overloading all of the relational operators, we give only the definitions of these member functions.

```
    //Overload the relational operators.
bool videoType::operator==(const videoType& right) const
{
    return (videoTitle == right.videoTitle);
}

bool videoType::operator!=(const videoType& right) const
{
    return (videoTitle != right.videoTitle);
}

bool videoType::operator <(const videoType& right) const
{
    return (videoTitle < right.videoTitle);
}
```

```
bool videoType::operator <=(const videoType& right) const
{
    return (videoTitle <= right.videoTitle);
}

bool videoType::operator >(const videoType& right) const
{
    return (videoTitle > right.videoTitle);
}

bool videoType::operator >=(const videoType& right) const
{
    return (videoTitle >= right.videoTitle);
}
```

VIDEO LIST The video list is maintained in a binary search tree. Therefore, we derive the `class videoBinaryTree` from the `class bSearchTreeType`. The definition of the `class videoBinaryTree` is as follows:

```
#include <iostream>
#include <string>
#include "binarySearchTree.h"
#include "videoType.h"

using namespace std;

class videoBinaryTree:public bSearchTreeType<videoType>
{
public:
    bool videoSearch(string title);
      //Function to search the list to see whether a particular
      //title, specified by the parameter title, is in stock.
      //Postcondition: Returns true if the title is found,
      //    false otherwise.
    bool isVideoAvailable(string title);
      //Function to determine whether at least one copy of a
      //particular video is in stock.
      //Postcondition: Returns true if at least one copy is in
      //    stock, false otherwise.
    void videoCheckOut(string title);
      //Function to check out a video, that is, rent a video.
      //Postcondition: copiesInStock is decremented by 1.
    void videoCheckIn(string title);
      //Function to check in a video returned by a customer.
      //Postcondition: copiesInStock is incremented by 1.
    bool videoCheckTitle(string title);
      //Function to determine whether a particular video is in
      //stock.
      //Postcondition: Returns true if the video is in stock,
      //    false otherwise.
```

```
    void videoUpdateInStock(string title, int num);
      //Function to update the number of copies of a video by
      //adding the value of the parameter num. The parameter title
      //specifies the name of the video for which the number of
      //copies is to be updated.
      //Postcondition: copiesInStock = copiesInStock + num

    void videoSetCopiesInStock(string title, int num);
      //Function to reset the number of copies of a video. The
      //parameter title specifies the name of the video for which
      //the number of copies is to be reset; the parameter num
      //specifies the number of copies.
      //Postcondition: copiesInStock = num

    void videoPrintTitle();
      //Function to print the titles of all the videos in stock.

private:
    void searchVideoList(string title, bool& found,
                         binaryTreeNode<videoType>* &current);
      //Function to search the video list for a particular video,
      //specified by the parameter title.
      //Postcondition: If the video is found, the parameter found
      //    is set to true, false otherwise. The parameter current
      //    points to the node containing the video.

    void inorderTitle(binaryTreeNode<videoType> *p);
      //Function to print the titles of all the videos in stock.
};
```

The definitions of the member functions of the `class videoBinaryTree` are similar to the ones given in Chapter 5. We only give the definitions of the functions `searchVideoList`, `inorderTitle`, and `videoPrintTitle`. (See Programming Exercise 12 at the end of the chapter.)

The function `searchVideoList` uses a search algorithm similar to the search algorithm for a binary search tree given earlier in this chapter. It returns `true` if the search item is found in the list. It also returns a pointer to the node containing the search item. Note that the function `searchVideoList` is a `private` member of the `class videoBinaryTree`. So the user cannot directly use this function in a program. Therefore, even though this function returns a pointer to a node in the tree, the user cannot directly access the node. The function `searchVideoList` is used only to implement other functions of the `class videoBinaryTree`. The definition of this function is as follows:

```
void videoBinaryTree::searchVideoList(string title, bool& found,
                                      binaryTreeNode<videoType>* &current)
{
    found = false;

    videoType temp;
```

```
    temp.setVideoInfo(title, "", "", "", "", "", 0);

    if (root == NULL)  //the tree is empty
        cout << "Cannot search an empty list. " << endl;
    else
    {
        current = root; //set current point to the root node
                        //of the binary tree
        found = false;  //set found to false

        while (!found && current != NULL) //search the tree
            if (current->info == temp)    //the item is found
                found = true;
            else if (current->info > temp)
                current = current->llink;
            else
                current = current->rlink;
    } //end else
}
```

Given a pointer to the root node of the binary tree containing the videos, the function `inorderTitle` uses the inorder traversal algorithm to print the titles of the videos. Notice that this function outputs only the video titles. The definition of this function is as follows:

```
void videoBinaryTree::inorderTitle(binaryTreeNode<videoType> *p)
{
    if (p != NULL)
    {
        inorderTitle(p->llink);
        p->info.printTitle();
        inorderTitle(p->rlink);
    }
}
```

The function `videoPrintTitle` uses the function `inorderTitle` to print the titles of all the videos in the store. The definition of this function is as follows:

```
void videoBinaryTree::videoPrintTitle()
{
    inorderTitle(root);
}
```

MAIN PROGRAM

The main program is the same as before. Here we give only the listing of this program. We assume that the name of the header file containing the definition of the `class videoBinaryTree` is `videoBinaryTree.h`, and so on.

```
//*****************************************************************
// Author: D.S. Malik
//
// This program illustrates how to use the classes videoType
// and videoBinaryTree to create and process a list of videos.
//*****************************************************************

#include <iostream>
#include <fstream>
#include <string>
#include "binarySearchTree.h"
#include "videoType.h"
#include "videoBinaryTree.h"

using namespace std;

void createVideoList(ifstream& infile,
                     videoBinaryTree& videoList);
void displayMenu();

int main()
{
    videoBinaryTree  videoList;
    int choice;
    char ch;
    string title;

    ifstream infile;

    infile.open("videoDat.txt");

    if (!infile)
    {
        cout << "The input file does not exist" << endl;
        return 1;
    }

    createVideoList(infile, videoList);
    infile.close();

    displayMenu();
    cout << "Enter your choice: ";
    cin >> choice;   //get the request
    cin.get(ch);
    cout << endl;

        //process the request
    while (choice != 9)
```

```
{
    switch(choice)
    {
    case 1:
        cout << "Enter the title: ";
        getline(cin, title);
        cout << endl;
        if (videoList.videoSearch(title))
            cout << "Title found." << endl;
        else
            cout << "The store does not carry this title."
                 << endl;
        break;

    case 2:
        cout << "Enter the title: ";
        getline(cin, title);
        cout << endl;
        if (videoList.videoSearch(title))
        {
            if (videoList.isVideoAvailable(title))
            {
                videoList.videoCheckOut(title);
                cout << "Enjoy your movie: " << title << endl;
            }
            else
                cout << "The video is currently out of stock."
                     << endl;
        }
        else
            cout << "The video is not in the store." << endl;
        break;

    case 3:
        cout << "Enter the title: ";
        getline(cin, title);
        cout << endl;
        if (videoList.videoSearch(title))
        {
            videoList.videoCheckIn(title);
            cout << "Thanks for returning " << title << endl;
        }
        else
           cout << "This video is not from our store." << endl;
        break;

    case 4:
        cout << "Enter the title: ";
        getline(cin, title);
        cout << endl;
```

```
                if (videoList.videoSearch(title))
                {
                    if (videoList.isVideoAvailable(title))
                        cout << "The video is currently in stock."
                             << endl;
                    else
                        cout << "The video is out of stock." << endl;
                }
                else
                    cout << "The video is not in the store." << endl;
                break;

            case 5:
                videoList.videoPrintTitle();
                 break;

            case 6:
                videoList.inorderTraversal();
                break;

            default: cout << "Invalid selection." << endl;
            }//end switch

            displayMenu();
            cout << "Enter your choice: ";
            cin >> choice;  //get the next request
            cin.get(ch);
            cout << endl;
        }//end while

        return 0;
    }

    void createVideoList(ifstream& infile, videoBinaryTree& videoList)
    {
        string title;
        string star1;
        string star2;
        string producer;
        string director;
        string productionCo;
        char   ch;
        int   inStock;

        videoType newVideo;

        getline(infile, title);
        while (infile)
        {
            getline(infile, star1);
            getline(infile, star2);
            getline(infile, producer);
```

```
        getline(infile, director);
        getline(infile, productionCo);
        infile >> inStock;
        infile.get(ch);
        newVideo.setVideoInfo(title, star1, star2, producer,
                              director, productionCo, inStock);
        videoList.insert(newVideo);

        getline(infile, title);
    }//end while
}//end createVideoList

void displayMenu()
{
    cout << "Select one of the following: " << endl;
    cout << "1: To check whether a particular video is in "
         << "the store" << endl;
    cout << "2: To check out a video" << endl;
    cout << "3: To check in a video" << endl;
    cout << "4: To see whether a particular video is in stock"
         << endl;
    cout << "5: To print the titles of all the videos" << endl;
    cout << "6: To print a list of all the videos" << endl;
    cout << "9: To exit" << endl;
}
```

B-Trees

In the previous sections of this chapter, we discussed how to build binary search trees and in particular AVL trees to effectively organize data dynamically and effectively search the data. However, the performance of the search depends on the height of the tree. In this section, we discuss B-trees in which the leaves are on the same level and are not too far from the root.

Definition: **(***m*-way search tree**)** An ***m*-way search tree** is a tree in which each node has at most m children, and if the tree is nonempty, it has the following properties:

1. Each node has the following form:

n	P_0	K_1	P_1	K_2	K_2	. . .	K_n	P_n

   where P_0, P_1, $P_2, \ldots, P_n$ are pointers to the subtrees of the node, K_1, $K_2, \ldots, K_n$ are keys such that $K_1 < K_2 < \ldots < K_n$, and $n \leq m - 1$.
2. All keys, if any, in the node to which P_i points are less than K_{i+1}.
3. All keys, if any, in the node to which P_i points are greater than K_i.
4. The subtrees, if any, to which each P_i points are m-way search trees.

Figure 11-24 shows a 5-way search tree.

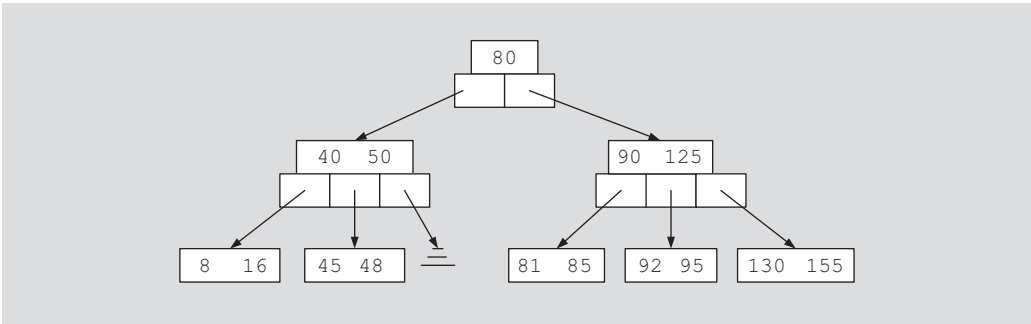


FIGURE 11-24 A 5-way search tree

Definition: **(**B-tree of order *m***)** A ***B*-tree of order *m*** is an *m*-way search tree that is either empty, or has the following properties:

1. All leaves are on the same level.
2. All internal nodes except the root have at most *m* (nonempty) children and at least $\lceil m/2 \rceil$ children. (Note that $\lceil m/2 \rceil$ denotes the ceiling of $m/2$.)
3. The root has at least 2 children if it is not a leaf, and at most *m* children.

Figure 11-25 shows a B-tree of order 5.

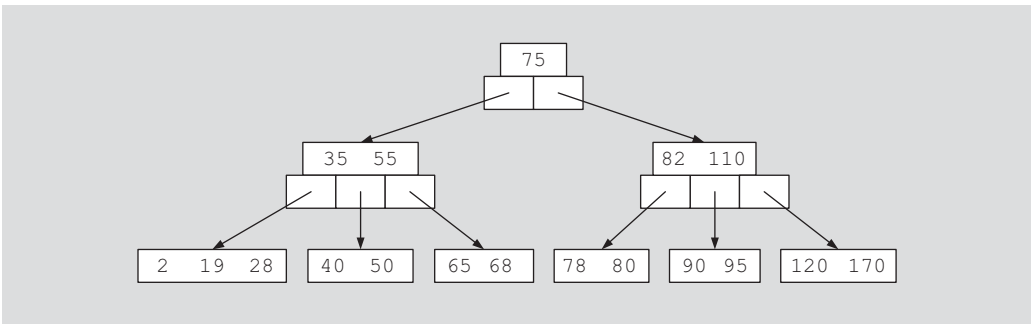


FIGURE 11-25 A B-tree of order 5

Note that the 5-way search tree in Figure 11-24 is not a B-tree of order 5.

The basic operations performed on a B-tree are search the tree, insert an item in the tree, delete an item from the tree, and traverse the tree. In the remainder of this section, we discuss how to implement some of these operations.

Before discussing these properties and describing the structure of a node and the class to implement the properties of a B-tree, we note the following: Until now, we have passed only data types as parameters to templates. Just like types (data types), constant expressions

can also be passed as parameters to templates. For example, consider the following `class template`:

```
template<class elemType, int size>
class listType
{
public:
    .
    .
    .
private:
    int maxSize;
    int length;
    elemType listElem[size];
};
```

This `class template` contains an array data member. The array element type and the size of the array are passed as parameters to the `class template`. To create a list of 100 components of `int` elements, we use the following statement:

```
listType<int, 100> intList;
```

Next, we give the definitions of the B-tree node and the class that implements the properties of a B-tree.

Each node should store the number of keys in the node, the records, and the pointer to subtrees. We use an array to store the records and an array to store the pointers to the subtrees. Thus, the definition of a B-tree node is as follows:

```
template <class recType, int bTreeOrder>
struct bTreeNode
{
    int recCount;
    recType list[bTreeOrder - 1];
    bTreeNode *children[bTreeOrder];
};
```

The class implementing the properties of a B-tree must, among others, implement the search, traversal, insertion, and deletion algorithms. The following class implements the basic properties of a B-tree as an ADT:

```
//*************************************************************
// Author: D.S. Malik
//
// class bTree
// This class specifies the basic operations to implement a
// B-tree.
//*************************************************************

template <class recType, int bTreeOrder>
class bTree
```

```
{
public:
    bool search(const recType& searchItem);
      //Function to determine if searchItem is in the B-tree.
      //Postcondition: Returns true if searchItem is found in the
      //    B-tree; otherwise, returns false.

    void insert(const recType& insertItem);
      //Function to insert insertItem in the B-tree.
      //Postcondition: If insertItem is not in the the B-tree, it
      //    is inserted in the B-tree.

    void inOrder();
      //Function to do an inorder traversal of the B-tree.

    bTree();
      //constructor

    //Add additional members as needed.

protected:
    bTreeNode<recType, bTreeOrder> *root;
};
```

Search

The function `search` searches the binary search tree for a given item. If the item is found in the binary search tree, it returns `true`; otherwise, it returns `false`. The search must start at the root node. Because, usually, there is more than one item in a node, we must search the array containing the data. Therefore, in addition to the function `search`, we also write the function `searchNode` that searches a node sequentially. If `item` is found, the function `searchNode` returns `true` and the location in the array where `item` is found. If `item` is not in the node, the function returns `false` and `location` points to either the first item that is larger than the search item or one past the last item in the node. The definitions of these functions are as follows:

```
template <class recType, int bTreeOrder>
void bTree<recType, bTreeOrder>::searchNode
                    (bTreeNode<recType, bTreeOrder> *current,
                     const recType& item,
                     bool& found, int& location)
{
    location = 0;

    while (location < current->recCount
           && item > current->list[location])
        location++;

    if (location < current->recCount
           && item == current->list[location])
        found = true;
    else
        found = false;
} //end searchNode
```

```
template <class recType, int bTreeOrder>
bool bTree<recType, bTreeOrder>::search(const recType& searchItem)
{
    bool found = false;
    int location;

    bTreeNode<recType, bTreeOrder> *current;

    current = root;

    while (current != NULL && !found)
    {
        searchNode(current, item, found, location);

        if (!found)
            current = current->children[location];
    }

    return found;
} //end search
```

Note that the function `searchNode` searches the node sequentially. However, because the data in the node is ordered, we can also use a binary search algorithm to search the node. We leave it as an exercise for you to modify the definition of the function `searchNode` so that it uses a binary search algorithm to search the node; see Programming Exercise 16 at the end of this chapter.

Traversing a B-Tree

As in the case of a binary tree, a B-tree can be traversed in three ways: inorder, preorder, and postorder. We only discuss the inorder traversal algorithm and leave the others as an exercise.

```
template <class recType, int bTreeOrder>
void bTree<recType, bTreeOrder>::inOrder()
{
    recInorder(root);
} // end inOrder

template <class recType, int bTreeOrder>
void bTree<recType, bTreeOrder>::recInorder
                      (bTreeNode<recType, bTreeOrder> *current)
{
    if (current != NULL)
    {
        recInorder(current->children[0]);
```

```
        for (int i = 0; i < current->recCount; i++)
        {
            cout << current->list[i] << " ";

            recInorder(current->children[i + 1]);
        }
    }
} //end recInorder
```

Insertion into a B-Tree

The general algorithm to insert an item in a B-tree is as follows.

Insert Algorithm: Search the tree to see if the key is already in the tree. If the key is already in the tree, output an error messsage. If the key is not in the tree, the search terminates at a leaf. The record is inserted into the leaf if there is room. If the leaf is full, split the node into two nodes and the median key is moved to the parent node. (Note that the median is determined by considering all the keys in the node and the new key to be inserted.) The splitting can propogate upward even as far as the root, causing the tree to increase in height.

Next, we illustrate how the insert algorithm works.

Figures 11-26 to 11-29 shows the insertion of items into an initially empty B-tree of order 5.

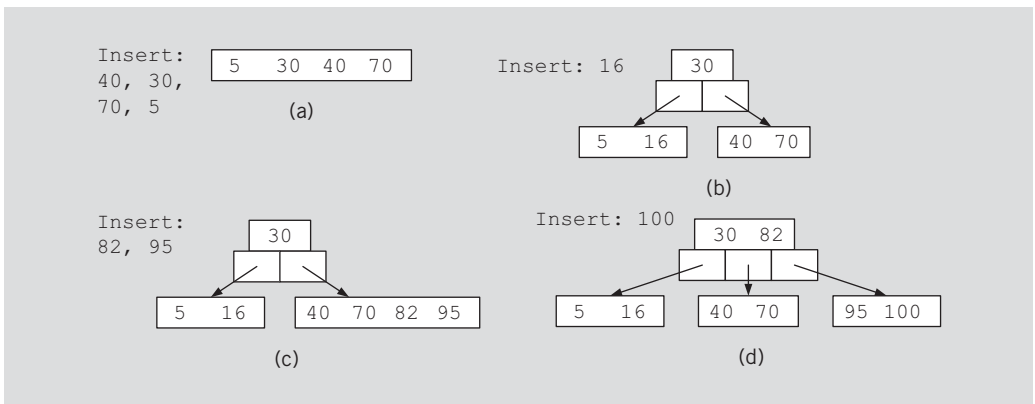


FIGURE 11-26 Item insertion into a B-tree of order 5

The insertion of `40`, `30`, `70`, and `5` is shown in Figure 11-26(a). The insertion of `16` requires to split the root node, causing the height of the tree to increase; see Figure 11-26(b). The insertion of `82` and `95` is shown in Figure 11-26(c). The next item inserted is `100`; see Figure 11-26(d). The item `100` is to be inserted in the right child of the root node. However, the right child of the root node is full. So we split this node and move the median key, which is `82`, to the parent node. Because the parent node is not full, we can insert `82` in the parent node; see Figure 11-26(d).

Figure 11-27 shows the insertion of 73, 54, 98, and 37.

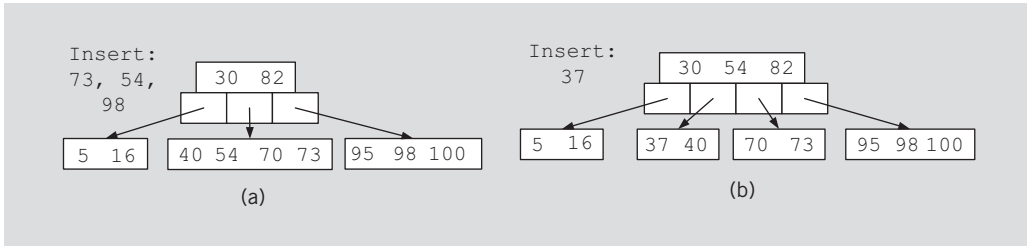


FIGURE 11-27 Insertion of 73, 54, 98, and 37

Note that 73, 54, and 98 are inserted without splitting any node; see Figure 11-27(a). However, the insertion of 37 requires the splitting of a node. The item 37 is to be inserted in the right child of 30. However, the right child of 30 is full, as shown in Figure 11-27(a). So we split the right child of 30, insert 37, and move the median key, which is 54, to the parent node. Because the parent node is not full, the median key 54 is inserted into the parent node; see Figure 11-27(b).

Figure 11-28 shows the insertion of 25, 62, 81, 150, and 79 into the B-tree of Figure 11-27(b).

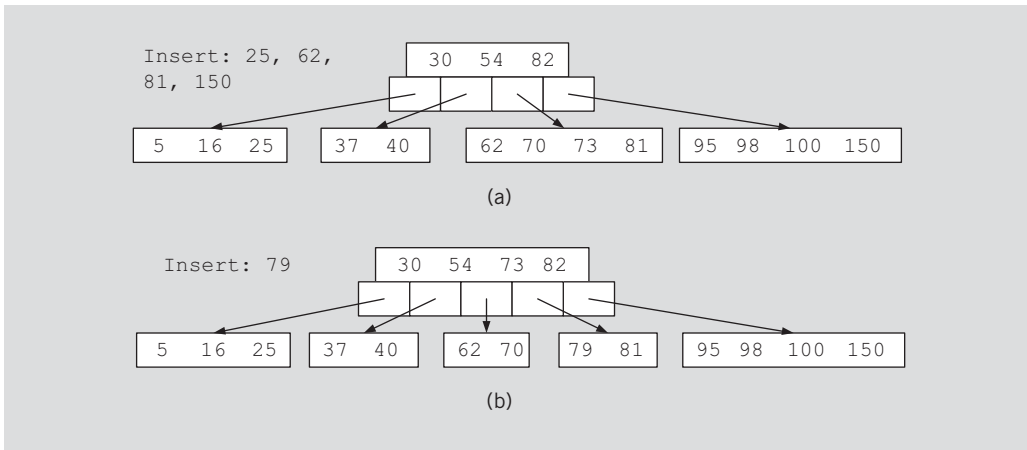


FIGURE 11-28 Insertion of 25, 62, 81, 150, and 79

Note that 25, 62, 81, and 150 are inserted without splitting any node; see Figure 11-28(a). However, the insertion of 79 requires the splitting of a node. The item 79 is to be inserted in the right child of 54. However, the right child of 54 is full; see Figure 11-28(a). So we split the right child of 54, insert 79, and move the median key, which is 73, to the parent node. Because the parent node is not full, the median key 73 is inserted into the parent node; see Figure 11-28(b).

Next, we insert 200, as shown in Figure 11-29.

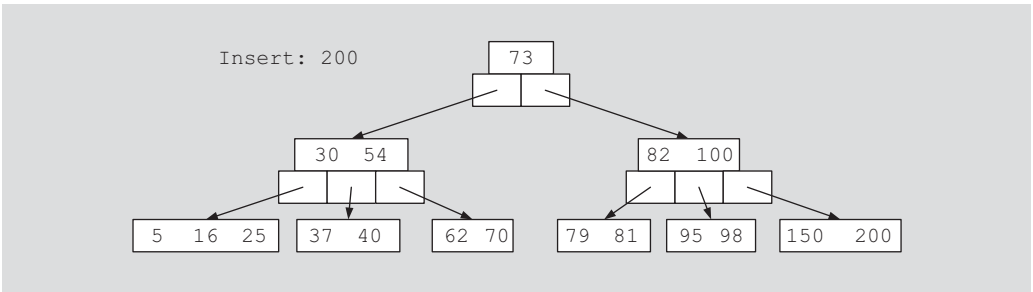


FIGURE 11-29 Insertion of 200

The item 200 is to be inserted in the right child of 82; see Figure 11-28(b). However, the right child of 82 is full. So we split the right child of 82, insert 200 in the node, and move the median key, which is 100, to the parent node. However, the parent node, which is the root node in Figure 11-28(b), is also full. So we split the parent node and move the median key, which is 73, to the new root node; see Figure 11-29. From Figure 11-29, it is clear that the height of the B-tree is increased.

From the previous discussion, it follows that to implement the insertion algorithm, we need algorithms to split a node, insert an item into a node, and move the median key to the parent node. Furthermore, because the insertion of an item might require the splitting of a node and moving the median key to the parent node, the simplest way to implement the insertion algorithm is to use recursion. To trigger the recursion, we will write another function, **insertBTree**. The definition of the function **insert**, which uses the function **insertBTree**, is as follows:

```
template <class recType, int bTreeOrder>
void bTree<recType, bTreeOrder>::insert(const recType& insertItem)
{
    bool isTaller = false;
    recType median;

    bTreeNode<recType, bTreeOrder> *rightChild;

    insertBTree(root, insertItem, median,
                rightChild, isTaller);

    if (isTaller) //the tree is initially empty or the root
                  //was split by the function insertBTree
    {
        bTreeNode<recType, bTreeOrder> *tempRoot;
        tempRoot = new bTreeNode<recType, bTreeOrder>;
        tempRoot->recCount = 1;
        tempRoot->list[0] = median;
```

```
        tempRoot->children[0] = root;
        tempRoot->children[1] = rightChild;

        root = tempRoot;
    }
} //insert
```

The function `insertBTree` recursively inserts an item into a B-tree. After inserting an item, it returns `true` if the height of the tree is to be increased. If the root node is to be split, this function splits the root node, sets `isTaller` to `true`, and sends the median key, `median`, and a pointer, `rightChild`, of the right child of `median` to the function `insert`. The function `insert` adjusts the root of the B-tree. This function has five parameters: `current`—a pointer to the B-tree in which to insert an item, `insertItem`—item to be inserted in the B-tree, `median`—to return the median key, `rightChild`—pointer to the right child of `median`, and `isTaller`—to indicate if the height of a B-tree is to be increased. In pseudocode, the algorithm is as follows:

```
if (current is NULL)
{
    Either the B-tree is empty or the search ends at an empty subtree.
    Set median to insertItem
    Set rightChild to NULL
    Set isTaller to true
}
else
{
    Call function searchNode to search the current node

    if insertItem is in the node
        output an error message
    else
    {
        call the function insertBTree with appropriate parameters
        if isTaller is true
           if current node is not full
               insert item into current node
           else
              call the function splitNode to split the node
    }
}
```

We leave it as an exercise for you to write the definition of the function `insertBTree`; see Programming Exercise 15 at the end of this chapter.

The function `insertNode` inserts an item in the node. Because the keys in the node are in order, the algorithm to insert a new item is similar to the `insertAt` function discussed in Chapter 3. The function has four parameters: `current`—a pointer to the node in which to insert the new item, `insertItem`—the item to be inserted, `rightChild`—a pointer to the right subtree of the item to be inserted, and `insertPosition`—the position in the array where to insert the item. The definition of this function is as follows:

```
template <class recType, int bTreeOrder>
void bTree<recType, bTreeOrder>::insertNode
                   (bTreeNode<recType, bTreeOrder> *current,
                    const recType& insertItem,
                    bTreeNode<recType, bTreeOrder>* &rightChild,
                    int insertPosition)
{
    int index;

    for (index = current->recCount; index > insertPosition;
                                    index--)
    {
        current->list[index] = current->list[index - 1];
        current->children[index + 1] = current->children[index];
    }

    current->list[index] = insertItem;
    current->children[index + 1] = rightChild;
    current->recCount++;
} //end insertNode
```

The function `splitNode` splits a node into two nodes and inserts the new item in the relevant node. It returns the median key and a pointer to the second half of the node. The parameter `current` points to the node to be split, `insertItem` is the item to be inserted, `newChild` is a pointer to the right child of the item to be inserted, `insertPosition` specifies the position where to insert the new item, after splitting the node the parameter `rightNode` returns a pointer to the right half of the node, and `median` returns the median key of the node.

```
template <class recType, int bTreeOrder>
void bTree<recType, bTreeOrder>::splitNode
                   (bTreeNode<recType, bTreeOrder> *current,
                    const recType& insertItem,
                    bTreeNode<recType, bTreeOrder>* rightChild,
                    int insertPosition,
                    bTreeNode<recType, bTreeOrder>* &rightNode,
                    recType &median)
{
    rightNode = new bTreeNode<recType, bTreeOrder>;

    int mid = (bTreeOrder - 1)  / 2;

    if (insertPosition <= mid) //new item goes in the first
                               //half of the node
    {
        int index = 0;
        int i = mid;
```

```
        while (i < bTreeOrder - 1)
        {
            rightNode->list[index] = current->list[i];
            rightNode->children[index + 1] =
                                  current->children[i + 1];
            index++;
            i++;
        }

        current->recCount = mid;
        insertNode(current, insertItem, rightChild,
                   insertPosition);
        (current->recCount)--;

        median = current->list[current->recCount];

        rightNode->recCount = index;
        rightNode->children[0] =
                  current->children[current->recCount + 1];
    }
    else  //new item goes in the second half of the node
    {
        int i = mid + 1;
        int index = 0;

        while (i < bTreeOrder - 1)
        {
            rightNode->list[index] = current->list[i];
            rightNode->children[index + 1] =
                                    current->children[i + 1];
            index++;
            i++;
        }

        current->recCount = mid;
        rightNode->recCount = index;

        median = current->list[mid];
        insertNode(rightNode, insertItem, rightChild,
                   insertPosition - mid - 1);
        rightNode->children[0] =
                  current->children[current->recCount + 1];
    }
} //splitNode
```

We leave it as an exercise for you to include the functions to insert an item in a B-tree and the functions to search and traverse a B-tree in the `class BTree`, and write a program to perform these operations on a B-tree; see Programming Exercise 15 at the end of this chapter.

Deletion from a B-Tree

To delete an item from a B-tree, we search the tree for the item to be deleted, say `deleteItem`. The following cases arise:

1. If `deleteItem` is not in the tree, output an appropriate error message.
2. If `deleteItem` is in the tree, find the node containing the `deleteItem`. If the node containing the `deleteItem` is not a leaf, its immediate predecessor (or successor) is in a leaf. So we can swap the immediate predecessor (or successor) with the `deleteItem` to move the `deleteItem` into a leaf. We consider the cases to delete an item from a leaf.
   a. If the leaf contains more than the minimum number of keys, delete the `deleteItem` from the leaf. (In this case, no further action is required.)
   b. If the leaf contains only the minimum number of keys, look at the sibling nodes that are adjacent to the leaf. (Note that the sibling nodes and the leaf have the same parent node.)
      i. If one of the sibling nodes has more than the minimum number of keys, move one of the keys from that sibling node to the parent and one key from the parent to the leaf, and then delete `deleteItem`.
      ii. If the adjacent siblings have only the minimum number of keys, then combine one of the siblings with the leaf and the median key from the parent. If this action does not leave the minimum number of keys in the parent node, this process of combining the nodes propogates upward, possibly as far as the root node, which could result in reducing the height of the B-tree.

Next, we illustrate how the deletion process works. Consider the B-tree of order 5 shown in Figure 11-30.

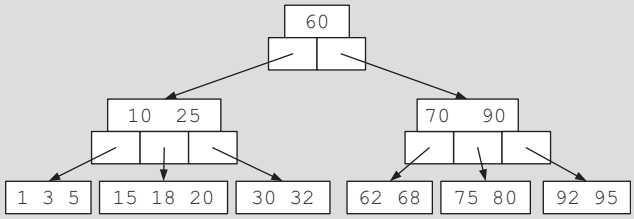


FIGURE 11-30 A B-tree of order 5

Let us delete `18` from this B-tree. Because `18` is in a leaf and the leaf has more than the minimum number of keys, we simply delete 18 from the leaf; see Figure 11-31.

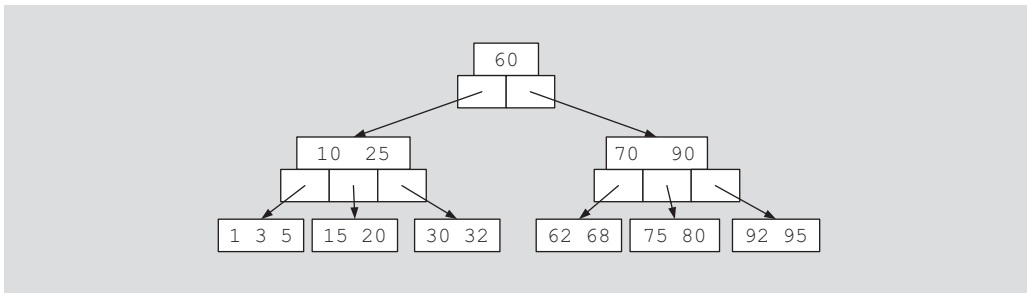


FIGURE 11-31 Deleting 18 from a B-tree of order 5

Next, let us delete 30. Figure 11-32 shows the B-tree before and after deleting 30.

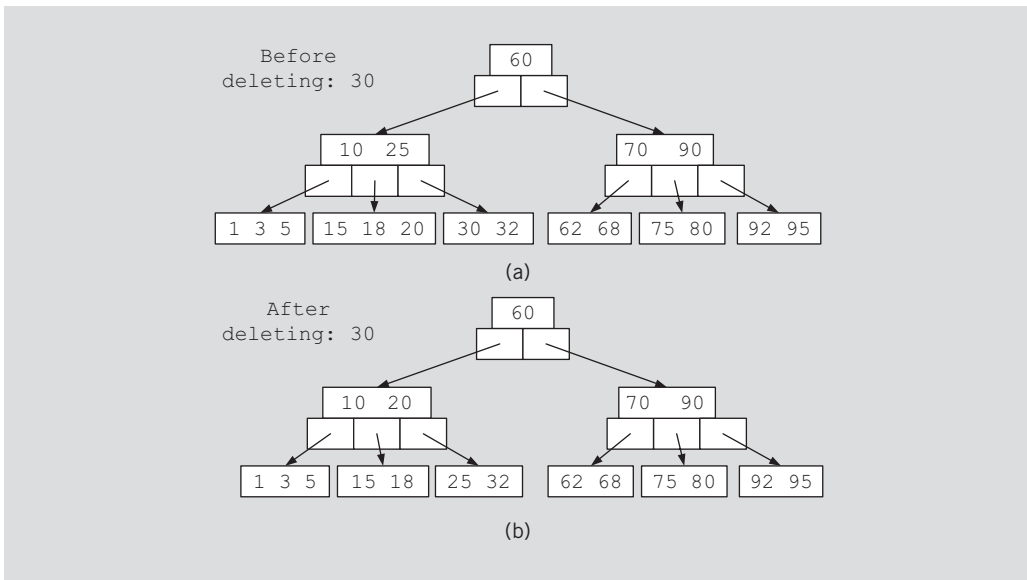


FIGURE 11-32 B-tree before and after deleting 30

The leaf containing 30 has only the minimum number of keys. However, its adjacent sibling has more than the minimum number of keys. So we move 20 from the adjacent sibling to the parent node and then move 25 from the parent node to the leaf; see Figure 11-32(b).

Next, let us delete 70. Figure 11-33 shows the process of deleting 70.

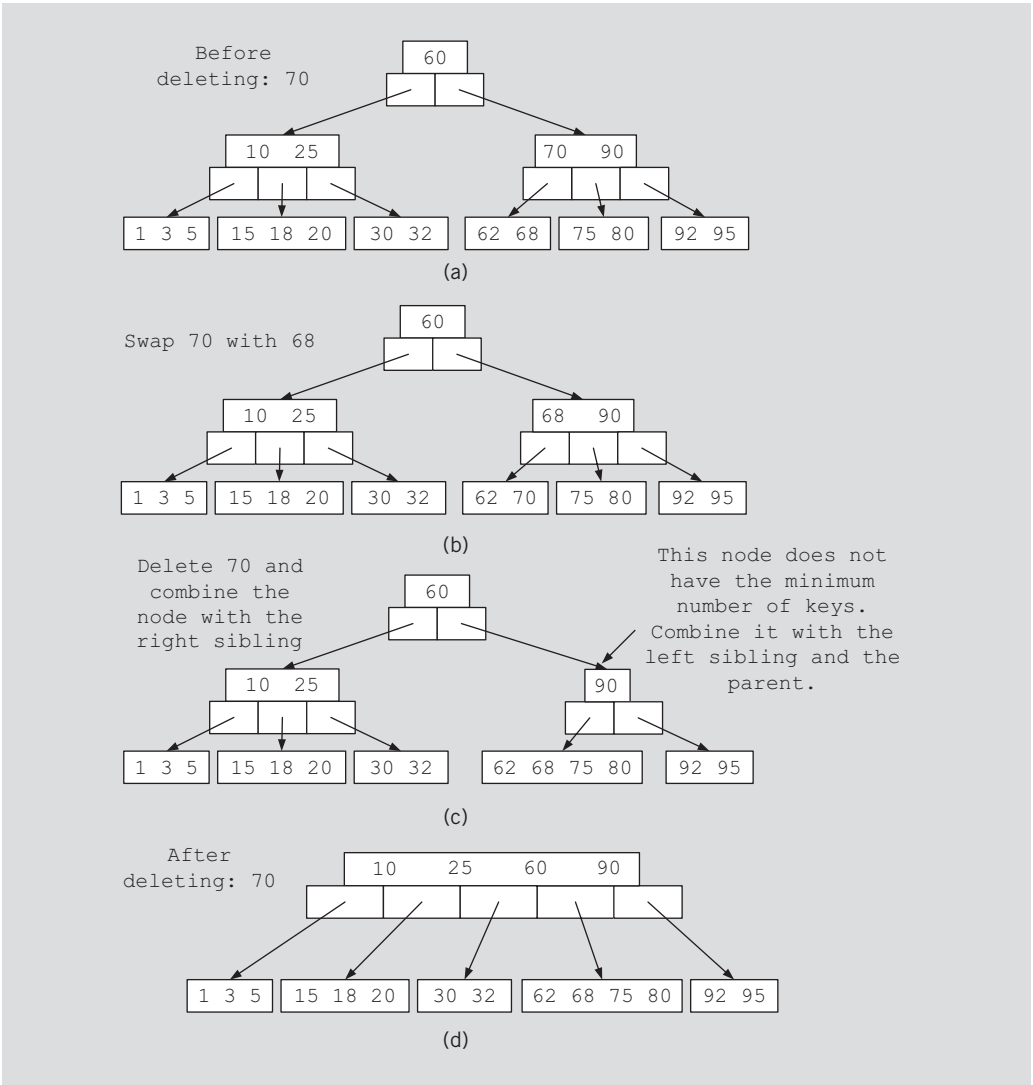


FIGURE 11-33 Deletion of 70 from the B-tree

The node containing 70 is not a leaf. So we swap 70 with its immediate predecessor, which is 68; see Figure 11-33(b). After deleting 70 from the leaf, because the leaf does not have the minimum number of keys, it is combined with its adjacent sibling; see Figure 11-33(c). However, this process does not leave the minimum number of keys in the parent node, which is 90. So we combine this node with its left sibling and their parent, which in this case is the root node; see Figure 11-33(d). Note that the deletion of 70 resulted in reducing the height of the B-tree.

We leave it as an exercise for you to develop the necessary algorithms to delete a record from a B-tree.

QUICK REVIEW

1. A binary tree is either empty or it has a special node called the root node. If the tree is nonempty, the root node has two sets of nodes, called the left and right subtrees, such that the left and right subtrees are also binary trees.
2. The node of a binary tree has two links in it.
3. A node in a binary tree is called a leaf if it has no left and right children.
4. A node U is called the parent of a node V if there is a branch from U to V.
5. A path from a node X to a node Y in a binary tree is a sequence of nodes $X_0, X_1, \ldots, X_n$ such that (a) $X = X_0$, $X_n = Y$ and (b) X_{i-1} is the parent of X_i for all $i = 1, 2, \ldots, n$. That is, there is a branch from X_0 to X_1, X_1 to $X_2, \ldots, X_{i-1}$ to $X_i, \ldots, X_{n-1}$ to X_n.
6. The level of a node in a binary tree is the number of branches on the path from the root to the node.
7. The level of the root node of a binary tree is `0`; the level of the children of the root node is `1`.
8. The height of a binary tree is the number of nodes on the longest path from the root to a leaf.
9. In an inorder traversal, the binary tree is traversed as follows: (a) Traverse the left subtree; (b) visit the node; (c) traverse the right subtree.
10. In a preorder traversal, the binary tree is traversed as follows: (a) Visit the node; (b) traverse the left subtree; (c) traverse the right subtree.
11. In a postorder traversal, the binary tree is traversed as follows: (a) Traverse the left subtree; (b) traverse the right subtree; (c) visit the node.
12. A binary search tree T is either empty or
    i. T has a special node called the *root* node.
    ii. T has two sets of nodes, L_T and R_T, called the left subtree and the right subtree of T, respectively.
    iii. The key in the root node is larger than every key in the left subtree and smaller than every key in the right subtree.
    iv. L_T and R_T are binary search trees.
13. To delete a node from a binary search tree that has both left and right nonempty subtrees, first its immediate predecessor is located, then the predecessor's `info` is copied into the node, and finally the predecessor is deleted.
14. A perfectly balanced binary tree is a binary tree such that
    i. The heights of the left and right subtrees of the root are equal.
    ii. The left and right subtrees of the root are perfectly balanced binary trees.

15. An AVL (or height-balanced) tree is a binary search tree such that
    i. The heights of the left and right subtrees of the root differ by at most 1.
    ii. The left and right subtrees of the root are AVL trees.
16. Let x be a node in a binary tree. x_l denotes the height of the left subtree of x; x_h denotes the height of the right subtree of x.
17. Let T be an AVL tree and x be a node in T. Then $|x_h - x_l| \leq 1$, where $|x_h - x_l|$ denotes the absolute value of $x_h - x_l$.
18. Let x be a node in the AVL tree T.
    a. If $x_l > x_h$, we say the x is left high. In this case, $x_l = x_h + 1$.
    b. If $x_l = x_h$, we say the x is equal high.
    c. If $x_h > x_l$, we say the x is right high. In this case, $x_h = x_l + 1$.
19. The balance factor of x, written $bf(x)$, is defined as $bf(x) = x_h - x_l$.
20. Let x be a node in the AVL tree T. Then,
    a. If x is left high, $bf(x) = -1$.
    b. If x is equal high, $bf(x) = 0$.
    c. If x is right high, $bf(x) = 1$.
21. Let x be a node in a binary tree. We say that node x violates the balance criteria if $|x_h - x_l| > 1$, that is, the heights of the left and right subtrees of x differ by more than 1.
22. Every node x in the AVL tree T, in addition to the data and pointers to the left and right subtrees, must keep track of its balance factor.
23. In an AVL tree, there are two types of rotations: left rotation and right rotation. Suppose that the rotation occurs at node, say x. If it is a left rotation, certain nodes from the right subtree of x move to its left subtree; the root of the right subtree of x becomes the new root of the reconstructed subtree. Similarly, if it is a right rotation at x, certain nodes from the left subtree of x move to its right subtree; the root of the left subtree of x becomes the new root of the reconstructed subtree.
24. A B-tree of order m is an m-way search tree that is either empty, or has the following properties: (1) All leaves are on the same level; (2) All internal nodes except the root have at most m (nonempty) children and at least $\lceil m/2 \rceil$ children. (Note that $\lceil m/2 \rceil$ denotes the ceiling of $m/2$.); (3) The root has at least 2 children if it is not a leaf, and at most m children.
25. To insert an item into a B-tree, search the tree to see if the record is already in the tree. If the record is already in the tree, output an error messsage. If the record is not in the tree, the search terminates at a leaf. The record is inserted into the leaf if there is room. If the leaf is full, split the node into two nodes and the median record is moved to the parent node. The splitting can propogate upward even as far as the root, causing the tree to increase in height.

EXERCISES

1. Mark the following statements as true or false.
   a. A binary tree must be nonempty.
   b. The level of the root node is `0`.
   c. If a tree has only one node, the height of this tree is `0` because the number of levels is `0`.
   d. The inorder traversal of a binary tree always outputs the data in ascending order.
2. There are 14 different binary trees with four nodes. Draw all of them.

The binary tree of Figure 11-34 is to be used for Exercises 3 through 8.

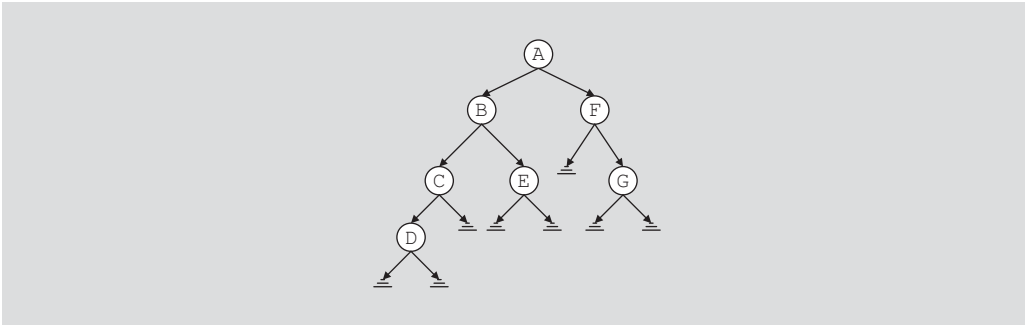


FIGURE 11-34 Binary tree for Exercises 3 through 8

3. Find L_A, the node in the left subtree of `A`.
4. Find R_A, the node in the right subtree of `A`.
5. Find R_B, the node in the right subtree of `B`.
6. List the nodes of this binary tree in an inorder sequence.
7. List the nodes of this binary tree in a preorder sequence.
8. List the nodes of this binary tree in a postorder sequence.

The binary tree of Figure 11-35 is to be used for Exercises 9 through 13.

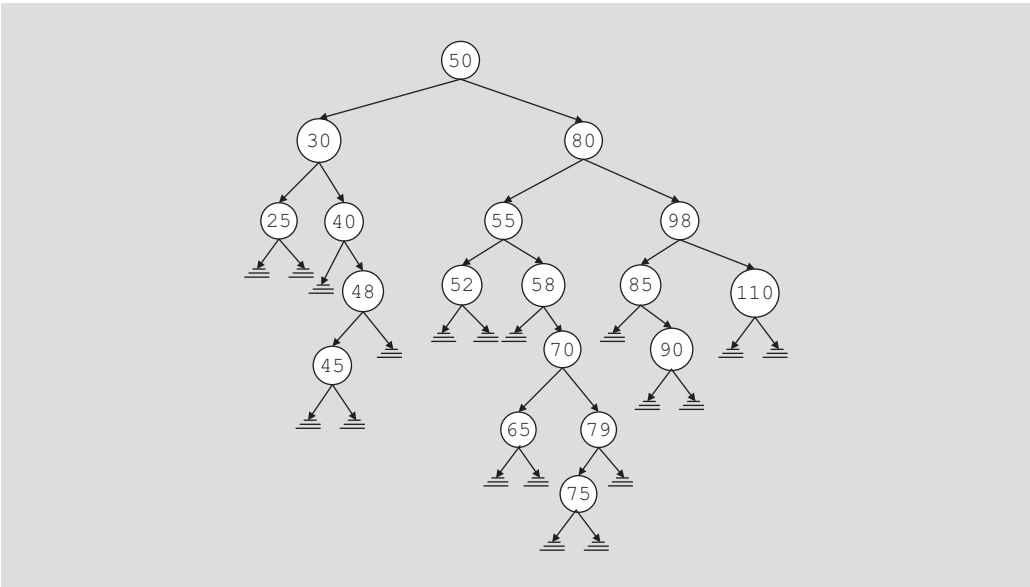


FIGURE 11-35 Binary tree for Exercises 9 through 13

9. List the path from the node with `info 80` to the node with `info 79`.
10. A node with `info 35` is to be inserted in the tree. List the nodes that are visited by the function `insert` to insert `35`. Redraw the tree after inserting `35`.
11. Delete node `52` and redraw the binary tree.
12. Delete node `40` and redraw the binary tree.
13. Delete nodes `80` and `58` in that order. Redraw the binary tree after each deletion.
14. Suppose that you are given two sequences of elements corresponding to the inorder sequence and the preorder sequence. Prove that it is possible to reconstruct a unique binary tree.
15. The nodes in a binary tree in preorder and inorder sequences are as follows:

```
preorder:   ABCDEFGHIJKLM
inorder:    CEDFBAHJIKGML
```

    Draw the binary tree.
16. Given the preorder sequence and the postorder sequence, show that it might not be possible to reconstruct the binary tree.

17. Insert `100` in the AVL tree of Figure 11-36. The resulting tree must be an AVL tree. What is the balance factor at the root node after the insertion?

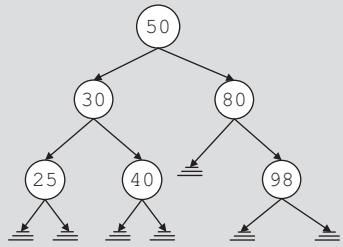


FIGURE 11-36 AVL tree for Exercise 17

18. Insert `45` in the AVL tree of Figure 11-37. The resulting tree must be an AVL tree. What is the balance factor at the root node after the insertion?

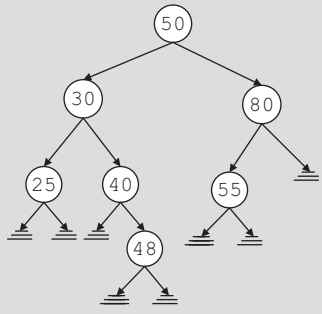


FIGURE 11-37 AVL tree for Exercise 18

19. Insert `42` in the AVL tree of Figure 11-38. The resulting tree must be an AVL tree. What is the balance factor at the root node after the insertion?

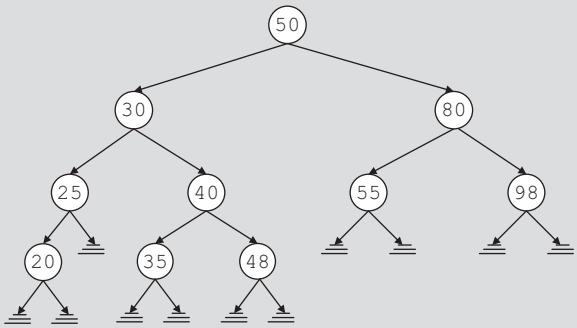


FIGURE 11-38 AVL tree for Exercise 19

20. The keys 24, 39, 31, 46, 48, 34, 19, 5, and 29 are inserted (in the order given) into an initially empty AVL tree. Show the AVL tree after each insertion.

The binary tree of Figure 11-39 is to be used for Exercises 21 to 23.

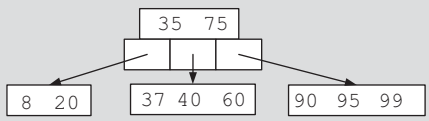


FIGURE 11-39 B-tree of order 5 for Exercises 21 to 23

21. Insert the keys 72, 30, and 50, in this order, into the B-tree of order 5 of Figure 11-39. Show the resulting tree.
22. Insert the keys 38, 45, 55, 80, and 85 into the B-tree of order 5 of Figure 11-39. Show the resulting tree.
23. Insert the keys 2, 30, 42, 10, 96, 15, 50, 82, and 98 into the B-tree of order 5 of Figure 11-39. Show the resulting tree.

The binary tree of Figure 11-40 is to be used for Exercises 24 to 27.

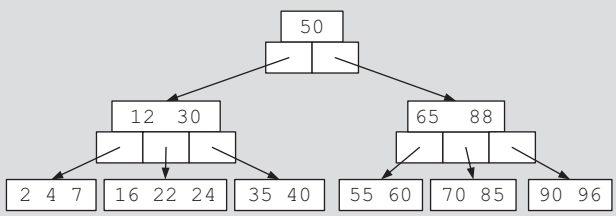


FIGURE 11-40 B-tree of order 5 for Exercises 24 to 27

24. Delete 7 from the B-tree of order 5 of Figure 11-40. Show the resulting tree.
25. Delete 40 from the B-tree of order 5 of Figure 11-40. Show the resulting tree.
26. Delete 88 from the B-tree of order 5 of Figure 11-40. Show the resulting tree.
27. Delete 12 from the B-tree of order 5 of Figure 11-40. Show the resulting tree.
28. Suppose that you have the keys 40, 30, 70, 5, 16, 82, 95, 100, 73, 54, 98, 37, 25, 62, 81, 150, 79, and 87.
    a. Insert these keys into an initially empty B-tree of order 5.
    b. Insert these keys into an initially empty B-tree of order 6.

PROGRAMMING EXERCISES

1. Write the definition of the function, `nodeCount`, that returns the number of nodes in a binary tree. Add this function to the `class binaryTreeType` and create a program to test this function.
2. Write the definition of the function, `leavesCount`, that takes as a parameter a pointer to the root node of a binary tree and returns the number of leaves in a binary tree. Add this function to the `class binaryTreeType` and create a program to test this function.
3. Write a function, `swapSubtrees`, that swaps all of the left and right subtrees of a binary tree. Add this function to the `class binaryTreeType` and create a program to test this function.
4. Write a function, `singleParent`, that returns the number of nodes in a binary tree that have only one child. Add this function to the `class binaryTreeType` and create a program to test this function. (Note: First create a binary search tree.)
5. Write a program to test various operations on a binary search tree.
6. a. Write the definition of the function to implement the nonrecursive postorder traversal algorithm.
   b. Write a program to test the nonrecursive inorder, preorder, and postorder traversal algorithms. (Note: First create a binary search tree.)
7. Write a version of the preorder traversal algorithm in which a user-defined function can be passed as a parameter to specify the visiting criteria at a node.
8. Write a version of the postorder traversal algorithm in which a user-defined function can be passed as a parameter to specify the visiting criteria at a node.
9. a. Write the definition of the class template that implements an AVL tree as an ADT. (You do not need to implement the delete operation.)
   b. Write the definitions of the member functions of the class that you defined in (a).
   c. Write a program to test various operations of an AVL tree.

10. Write a function that inserts the nodes of a binary tree into an ordered linked list. Also write a program to test your function.
11. Write a program to do the following:
    a. Build a binary search tree, T_1.
    b. Do a postorder traversal of T_1 and while doing the postorder traversal, insert the nodes into a second binary search tree T_2.
    c. Do a preorder traversal of T_2 and while doing the preorder traversal, insert the node into a third binary search tree T_3.
    d. Do an inorder traversal of T_3.
    e. Output the heights and the number of leafs in each of the three binary search trees.
12. Write the definitions of the functions of the `class videoBinaryTree` not given in the Programming Example Video Store. Also write a program to test the video store program.
13. (**Video Store Program**) In Programming Exercise 14 in Chapter 5, you were asked to design and implement a class to maintain customer data in a linked list. Because the search on a linked list is sequential and, therefore, can be time consuming, design and implement the `class customerBTreeType` so that this customer data can be stored in a binary search tree. The `class customerBTreeType` must be derived from the `class bSearchTreeType` as designed in this chapter.
14. (**Video Store Program**) Using classes to implement the video data, video list data, customer data, and customer list data, as designed in this chapter and in Programming Exercises 12 and 13, design and complete the program to put the video store into operation.
15. Write the definition of the function `insertBTree` to recursively insert a record into a B-tree. Also write a program to test various operations on a B-tree.
16. Rewrite the definition of the function `searchNode` of the class B-tree so that it uses a binary search to search the node. Also write a program to test various operations on a B-tree.

This page intentionally left blank